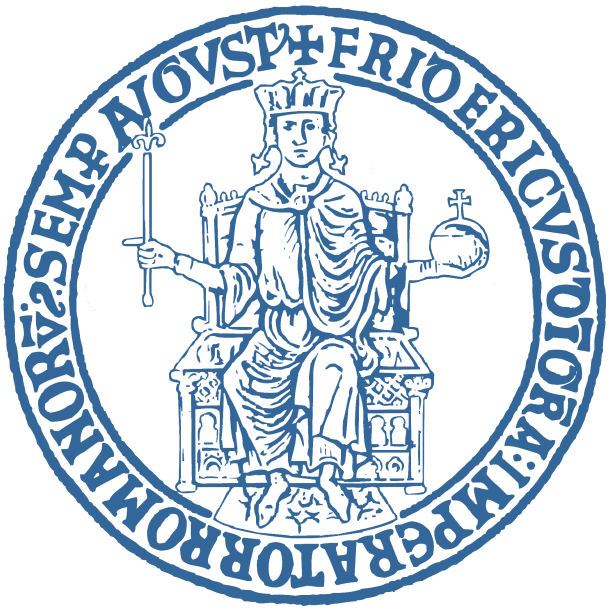
Università degli Studi di Napoli



Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione

Corso di Laurea Magistrale in Informatica

Knowledge-based Recommender System: a MovieLens Small application

Professore
Luigi Sauro

Autori
Mario Gabriele Carofano
N97000437

Oleksandr Sosovskyy
N97000464

A.A. 2024 – 2025

Abstract

Il presente lavoro descrive lo sviluppo di un Knowledge-Based Recommender System nel dominio dei film, progettato a partire dal dataset MovieLens Small e arricchito mediante l'acquisizione di informazioni da DBpedia tramite query SPARQL. L'obiettivo è fornire raccomandazioni basate sulle caratteristiche intrinseche dei film, integrando i loro metadati da fonti esterne. Il sistema è stato realizzato in Python e incorpora tecniche di Multi-Armed Bandit per affrontare il problema di exploration vs exploitation, garantendo un bilanciamento dinamico tra novità e accuratezza nelle raccomandazioni. La valutazione sperimentale, condotta attraverso metriche consolidate, evidenzia i punti di forza e le aree di miglioramento. I risultati ottenuti confermano la validità dell'approccio e aprono la strada a futuri sviluppi, volti a potenziare la scalabilità, l'integrazione con altri dataset e la spiegabilità delle raccomandazioni.

Indice

1 Introduzione **2**

1.1 Traccia del progetto 3

2 Background teorico **4**

3 Architettura del sistema **6**

3.1 Struttura generale 6

3.2 Acquisizione dei dati 7

3.2.1 Fasi del processo di acquisizione dei dati 7

3.2.2 Descrizione dei file CSV generati 9

3.3 Preprocessing dei dati 10

3.3.1 Vettorizzazione dei metadati 11

3.3.2 Misurare la similarità tra i film 12

3.3.3 Predizione dei rating assenti 13

3.4 Sistema di raccomandazione 13

3.4.1 Profilazione utente content-based 15

3.4.2 MAB: Softmax Exploration 17

3.5 Il sistema client-server per la comunicazione con l'utente 19

3.5.1 Comunicazione tramite API REST 19

3.5.2 Rappresentazione dei dati 23

3.5.3 Gestione dell'interfaccia utente 24

4 Spiegabilità delle raccomandazioni **30**

4.1 Scelte di progettazione 30

4.2 Metriche utilizzate 30

4.3 Casi utente con screenshot 31

4.3.1 Utente n. 3 31

4.3.2 Utente n. 5 32

4.3.3 Utente n. 37 33

5 Valutazione del sistema **35**

5.1 LensKit 35

5.2 Pipeline di valutazione 35
5.3 Risultati Sperimentali 39

6 Conclusioni **40**
6.1 Sintesi dei Risultati 40
6.2 Limitazioni e sviluppi futuri 40

Bibliografia **42**

Elenco delle figure

1 I vari tipi di sistemi di raccomandazione 5

2 Architettura client-server del sistema 6

3 Il package **DataCollection** dedicato all'acquisizione dei dati 7

4 Il package **DataProcessing** dedicato all'elaborazione dei dati 10

5 Distribuzione della popolarità delle feature. L'asse X mostra i percentili delle feature. L'asse Y logaritmico mostra il numero di film associati a ciascuna percentuale feature. 12

6 Il package **Backend** incapsula il motore di raccomandazione, le API REST e il sistema di valutazione 14

7 Il package **Services** adibito alla comunicazione con il server 20

8 Il package **Model** descrive la rappresentazione dei dati provenienti dal server . . 23

9 Il package **View** nel quale sono implementate le schermate dell'applicativo client 24

10 La schermata principale offerta da **HomeRoute** 25

11 Le statistiche di una feature in **HomeRoute** 25

12 La schermata di login offerta da **LoginRoute** 26

13 La schermata dei dettagli di un film offerta da **MovieInfoRoute** 26

14 La schermata di query al database offerta da **MovieQueryRoute** 27

15 Dettagli tecnici di una raccomandazione dal widget **RecSysMovieCard** in **MovieQueryRoute** 27

16 Ordinamento dei risultati della query in **MovieQueryRoute** 28

17 La schermata delle impostazioni offerta da **SettingsRoute** 28

18 Una schermata di errore offerta da **ErrorScreen** 29

19 Caso utente n. 3 - Esempio 1 31

20 Caso utente n. 3 - Esempio 2 32

21 Caso utente n. 5 - Esempio 1 32

22 Caso utente n. 5 - Esempio 2 33

23 Caso utente n. 37 - Valutazioni 33

24 Caso utente n. 37 34

1 Introduzione

Il presente lavoro illustra lo sviluppo, il funzionamento e la valutazione di un sistema di raccomandazione per il dominio cinematografico. Nella fattispecie, la soluzione proposta è un recommender system content-based , in cui ogni raccomandazione dipende solo dalle caratteristiche dei film, opportunamente pesate dalle valutazioni precedenti dello stesso utente di film simili per suddette caratteratteristiche.

Questo approccio *knowledge-based* è tanto più performante quanto più è ricco il dataset di base, *MovieLens Small*, e le altre fonti con cui è arricchito. È stato selezionato *DBpedia*, un knowledge graph derivato da Wikipedia, come fonte primaria per l'estrazione di ulteriori metadati semantici (attori, registi, generi, ecc.) tramite query *SPARQL*.

Oltre all'arricchimento dei metadati, un altro requisito fondamentale del lavoro è l'implementazione di un opportuno algoritmo *Multi-Armed Bandit (MAB)* per bilanciare efficacemente il dilemma tra *exploitation* (consigliare i film dai migliori rating predetti) ed *exploration* (consigliare i film dai rating predetti non ottimali così da stupire l'utente e da aumentare la conoscenza sulle sue preferenze).

L'architettura software presenta un server, sviluppato interamente in Python, che ospita il motore di raccomandazione e gestisce l'intera pipeline di raccolta, modellazione ed elaborazione dei dati. Il server comunica con il client, implementato in Dart tramite il framework Flutter, responsabile dell'interfaccia utente e della visualizzazione delle raccomandazioni personalizzate e diversificate per l'utente che si vuole valutare. Il nostro sistema include, inoltre, un modulo Python per la valutazione del motore di raccomandazione, basata sul confronto con algoritmi allo stato dell'arte.

Il report si sviluppa nelle seguenti sezioni:

- La sezione 1 - ***Introduzione*** - a pagina 2 comincia il report con una panoramica sul contesto dei sistemi di raccomandazione, presentando le motivazioni che hanno portato allo sviluppo del progetto. Inoltre, vengono delineati gli obiettivi generali del lavoro e il contributo atteso.

- La sezione 2 - ***Background teorico*** - a pagina 4 si introduce il concetto di recommender system, definendone le caratteristiche fondamentali. Viene fornita una panoramica delle principali tipologie di sistemi di raccomandazione, con particolare attenzione ai modelli content-based, illustrandone i principi e le differenze.

- La sezione 3 - ***Architettura del sistema*** - a pagina 6 si descrive l'architettura del sistema, illustrando la struttura generale della comunicazione client-server tramite un diagramma a blocchi. Inoltre, si descrive l'intera pipeline di collezione dei dati, preprocessing e algoritmo di raccomandazione proposto.

- La sezione 5 - ***Valutazione del sistema*** - a pagina 35 si illustra la valutazione del sistema, descrivendo le metodologie adottate e gli esperimenti condotti. Vengono presentate le metriche impiegate, seguite dalla presentazione dei risultati ottenuti e da un'analisi di questi, volta a evidenziare punti di forza e debolezza del sistema.

- La sezione 6 - ***Conclusioni e sviluppi futuri*** - a pagina 40 si conclude il report sintetizzando i risultati ottenuti e mettendo in evidenza i punti chiave del sistema sviluppato. Vengono inoltre delineate le possibili estensioni e le future direzioni di sviluppo.

1.1 Traccia del progetto

Di seguito viene presentata la traccia del progetto, che illustra gli obiettivi generali e la struttura complessiva del lavoro.

> Realizzare un Recommender System sul dominio dei film con le seguenti caratteristiche:
>
> - Deve basarsi sul dataset *MovieLens Small* [1];
> - Opzionalmente si possono consultare anche altri dataset come quelli elencati in [2];
> - Il recommender system deve sfruttare le caratteristiche dei film, raccogliendo ed elaborando le informazioni rilevanti da *DBpedia* mediante opportune query *SPARQL*;
> - Opzionalmente può usare i tag presenti nei vari dataset (ad esempio *genres* in MovieLens);
> - Opzionalmente possono essere usate o definite ontologie sul dominio di interesse;
> - Inoltre il framework sviluppato deve utilizzare tecniche di *Multi-Armed Bandit (MAB)* per mediare il problema di *exploration vs exploitation*;
> - Parte integrante del progetto è la metodologia di verifica sperimentale, che deve fornire indicazioni chiare sull'accuratezza del framework proposto;
> - Il progetto può ispirarsi a lavori di ricerca già pubblicati, che devono essere opportunamente citati;
> - Il progetto deve essere realizzato in *Python* o *Java*.

2 Background teorico

Un sistema di raccomandazione (anche detto motore di raccomandazione) è un software progettato per supportare l'utente nel proccesso decisionale, filtrando e selezionando un sottoinsieme rilevante di opzioni da un insieme molto ampio di alternative. Può essere utilizzato in molteplici contesti, tra cui e-commerce, social network o nel campo dell'intrattenimento (es. libri, musica, film, video, notizie, ...).

Un recommender system mira ad aumentare la user satisfaction offrendo raccomandazioni che rispondano efficacemente ai bisogni e alle preferenze dell'utente, sfruttando dati relativi a preferenze esplicite o implicite degli utenti e a caratteristiche degli oggetti da raccomandare.

$$rec(P) = \arg\max_{i \in Items} E_P[u(i)] \tag{1}$$

dove:

- $rec(P)$: la raccomandazione calcolata dal sistema per l'utente, in base alla distribuzione P delle preferenze dell'utente.
- $Items$: insieme di tutti gli oggetti possibili che il sistema può raccomandare.
- $u(i)$: funzione di utilità che rappresenta il grado di soddisfazione dell'utente per l'item i.
- $E_P[u(i)] = \int P(u)\, u(i)\, du$: valore atteso dell'utilità dell'item i secondo la distribuzione delle preferenze P.
- $P(u)$: distribuzione di probabilità sulle possibili utilità u che l'utente può associare agli item.

Parallelamente, un aspetto cruciale nella progettazione dei sistemi di raccomandazione è la minimizzazione della user regret, ovvero la sensazione di rimpianto che un utente può provare quando, a posteriori, percepisce che una delle alternative non suggerite avrebbe potuto portare a una maggiore utilità o soddisfazione.

$$rec(P) = \arg\min_{i \in Items} E_P[u(best_item_u) - u(i)] \tag{2}$$

dove:

- $best_item_u$: l'item ottimale, cioè quello che massimizza la funzione di utilità u per una data distribuzione delle preferenze P dell'utente.

Le funzioni di raccomandazione possono essere concepite sia in ottica utility-based, che mira a massimizzare l'utilità attesa, sia in ottica regret-based, che cerca di minimizzare l'atteso rimpianto rispetto alla migliore opzione disponibile.

I sistemi di raccomandazione si differenziano per metodologia e fonti informative.

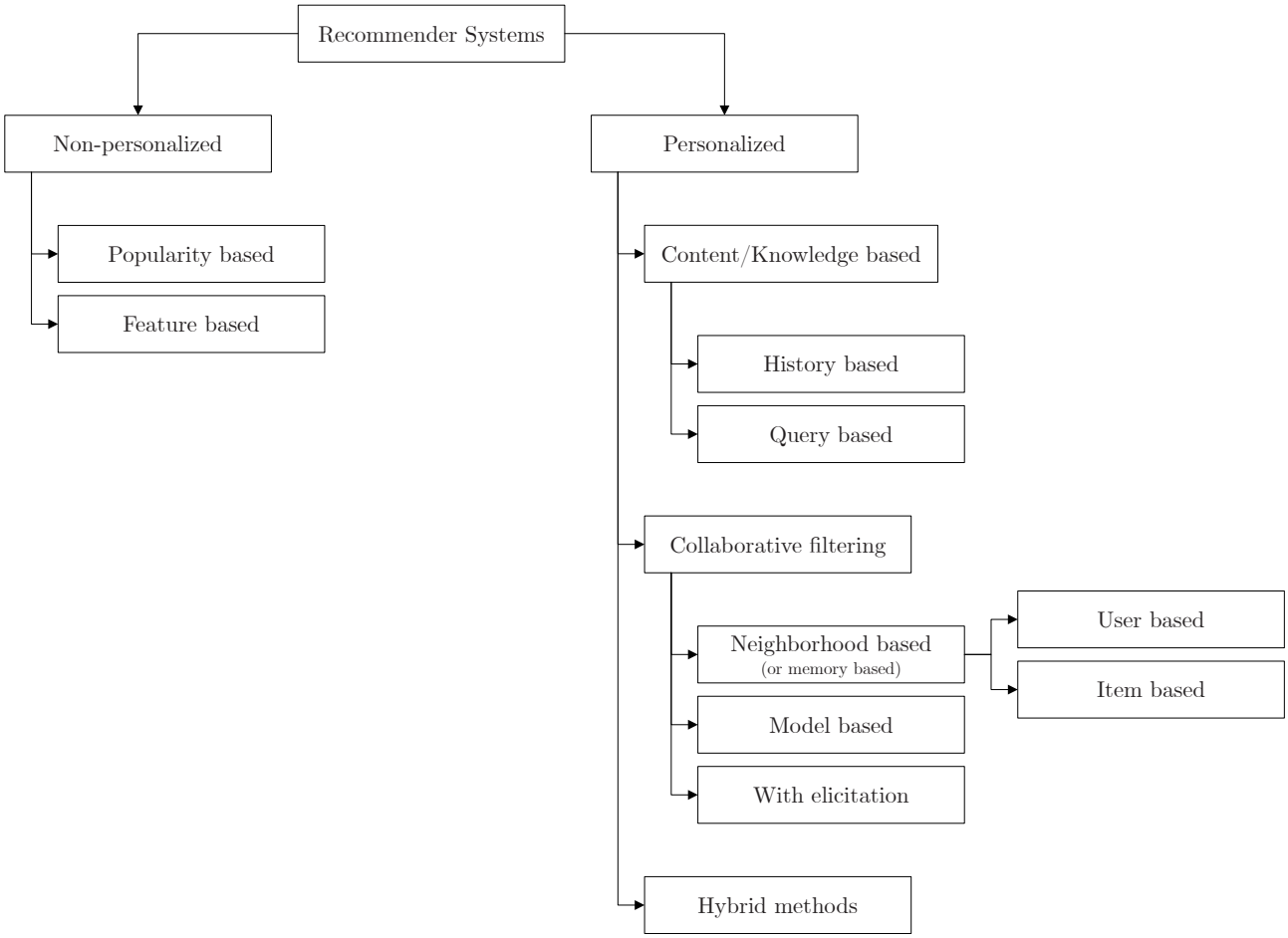


Figura 1: *I vari tipi di sistemi di raccomandazione*

Il presente lavoro si focalizza esclusivamente sui modelli content / knowledge based. Questi modelli utilizzano i soli attributi degli utenti e/o degli item (es. features, tag, ...) che li descrivono per generare raccomandazioni, a differenza dei sistemi con collaborative filtering che si basano sull'interazione tra essi. In particolare:

History based Il sistema tende a favorire le caratteristiche scelte più frequentemente o di recente, proponendo item simili a quelli che sono stati apprezzati in passato.

Query based Prescinde dai dati storici di valutazione, e genera raccomandazioni basandosi su requisiti esplicitamente dichiarati dall'utente.

3 Architettura del sistema

La presente sezione descrive l'architettura del sistema di raccomandazione sviluppato, mostrandone la struttura generale e analizzando nel dettaglio i principali moduli funzionali. In apertura, viene presentato un diagramma a blocchi che rappresenta l'interazione tra le diverse componenti software e la comunicazione tra client e server. Successivamente, ciascun modulo viene illustrato in una sottosezione dedicata, evidenziandone il ruolo all'interno del sistema con le principali funzionalità implementate.

3.1 Struttura generale

L'architettura segue un modello client-server. Il server, realizzato in Python, ospita il motore di raccomandazione e gestisce l'intera pipeline di elaborazione, dalla raccolta dei dati fino alla generazione delle raccomandazioni. Il client, realizzato in Dart tramite il framework Flutter, è destinato alla gestione dell'interfaccia utente e alla visualizzazione delle raccomandazioni generate. La comunicazione client-server avviene tramite interfacce di tipo API REST, che garantiscono flessibilità e scalabilità nell'interscambio di dati.

La figura 2 a pagina 6 illustra l'architettura client-server del sistema di raccomandazione tramite un class diagram. È stato scelto questo tipo di diagramma in quanto è particolarmente adatto per evidenziare la struttura modulare del sistema, mostrando chiaramente i diversi componenti (moduli, script, file, ...) che appartengono sia al Client che al Server, e le dipendenze esistenti tra di essi.

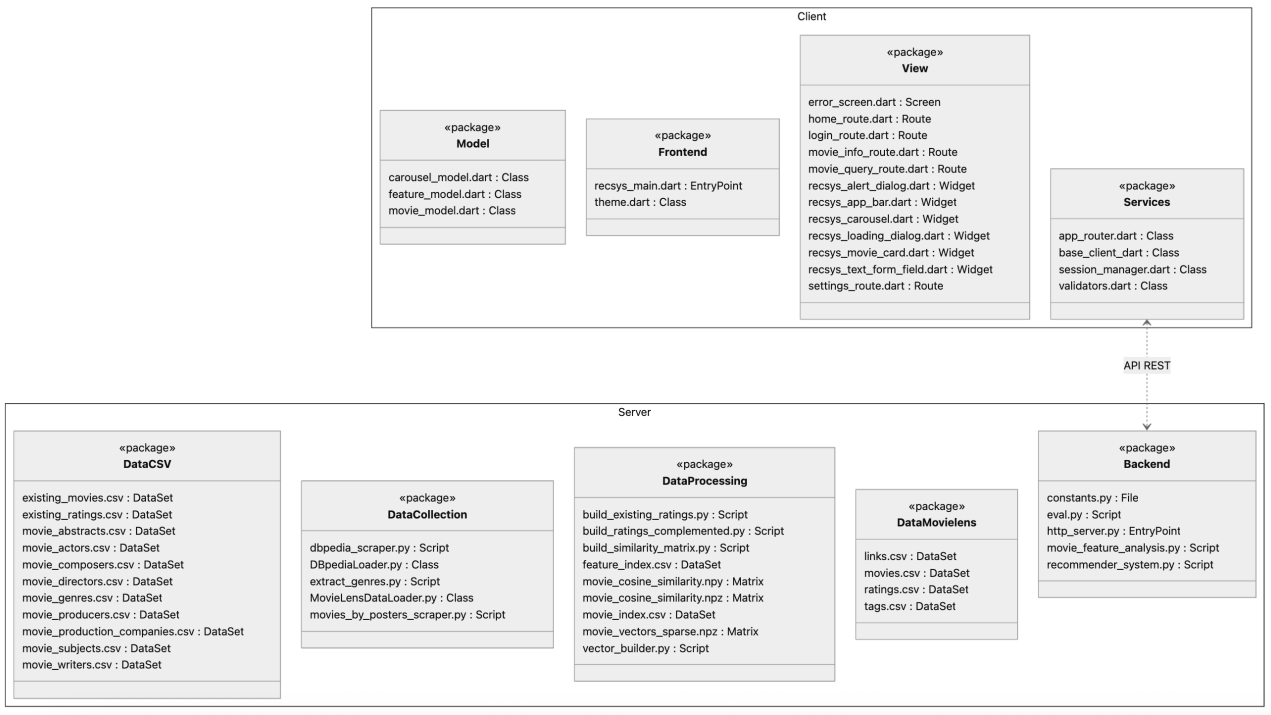


Figura 2: *Architettura client-server del sistema*

3.2 Acquisizione dei dati

Si assume che le prestazioni di un algoritmo di raccomandazione content-based dipendano dalla quantità, oltre che dalla qualità, dei dati esistenti. Per questo motivo il dataset di base, *MovieLens Small*, è stato arricchito con metadati esterni. Questo processo, noto come *data enrichment*, sfrutta due fonti di conoscenza principali: *DBpedia* e *Wikipedia*.

DBpedia [3] nasce da un progetto che mira a estrarre informazioni strutturate da Wikipedia, così da renderle disponibili sul web ed estraibili tramite un linguaggio di query specifico chiamato *SPARQL*. In pratica, il testo in linguaggio naturale delle pagine di Wikipedia è trasformato in un grande *knowledge graph*: entità come film, attori o registi sono rappresentate come nodi, e le relazioni tra di esse (es. "un attore ha recitato in un film") sono rappresentate come archi.

La figura 3 [1] rappresenta l'architettura del modulo *DataCollection*, collocato all'interno del server del sistema di raccomandazione. Si evidenziano i principali script impiegati per la raccolta e la generazione dei dati, quali `dbpedia_scraper.py`, `extract_genres.py`, `movies_by_posters_scraper.py`, oltre a moduli di utility come `DBpediaLoader.py` e `MovieLensDataLoader.py`. Inoltre, sono documentati anche i principali file generati dall'esecuzione di tali script (es. `movie_actors.csv`, `movie_composers.csv`, ...).

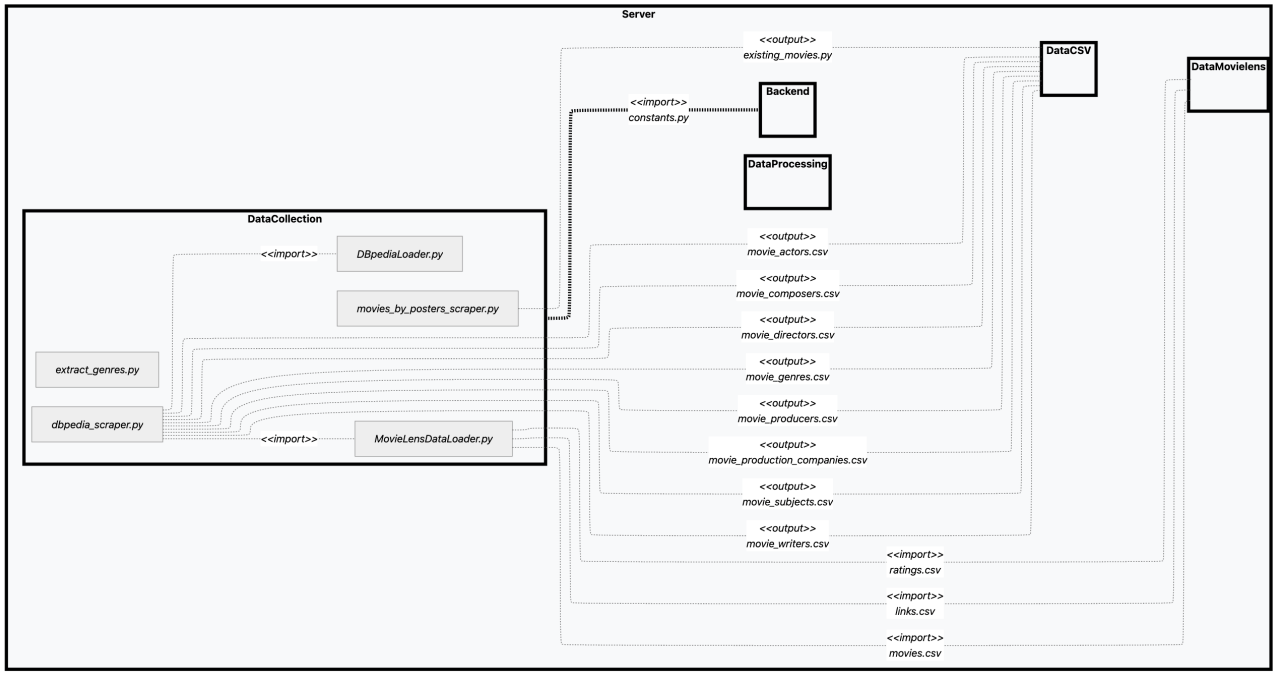


Figura 3: *Il package* ***DataCollection*** *dedicato all'acquisizione dei dati*

3.2.1 Fasi del processo di acquisizione dei dati

Il flusso di lavoro per l'arricchimento dei dati consiste dei seguenti passaggi:

[1] In tutte le figure che rappresentano l'architettura di sistema si utilizza la connessione in neretto per intendere una lettura / scrittura da parte di tutti i file del package.

1. **Estrazione e Normalizzazione del Titolo**: Il primo passo consiste nel preparare i titoli dei film presenti nel dataset MovieLens per renderli compatibili con la struttura di DBpedia. Ogni titolo viene "normalizzato" attraverso alcune semplici trasformazioni euristicamente determinate:

   - Viene rimosso l'anno di produzione (es. "(2008)").
   - Gli articoli come "The", "A", "An", se posti alla fine del titolo, vengono spostati all'inizio.
   - Gli spazi vengono sostituiti da underscore (_) per creare un formato URI valido.

   Ad esempio, il titolo "Dark Knight, The (2008)" diventa semplicemente "The_Dark_Knight". Questo passaggio è fondamentale perché la risorsa corrispondente in DBpedia venga trovata.

2. **Identificazione della risorsa in DBpedia**: Una volta normalizzato il titolo, il sistema tenta di trovare l'URI univoco che rappresenta il film su DBpedia. Poiché un titolo potrebbe non essere univoco (esistono film omonimi), vengono testati diversi candidati, come ad esempio `Nome_Film_(film)` oppure `Nome_Film_(Anno_film)`.

   Per verificare che un URI candidato sia corretto, viene eseguita una query SPARQL di tipo `ASK`. Questa query non restituisce dati, ma semplicemente un valore booleano (*true/false*), confermando se la risorsa esiste con il titolo provato ed è effettivamente classificata come un film (Verificando la presenza di proprietà tipiche come `dbo:director` o `dbo:starring`).

3. **Estrazione dei metadati con SPARQL**: Dopo aver ottenuto l'URI del film valido, vengono eseguite una serie di query *SPARQL SELECT* in cascata per estrarre i metadati di interesse (attori, registi, generi, riassunto, ecc.).

   Ad esempio, una di queste query potrebbe essere la seguente (usata per estrarre gli attori di "Pulp Fiction"):

```
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT DISTINCT ?actorName
WHERE {
  <http://dbpedia.org/resource/Pulp_Fiction> dbo:starring ?actorResource .
  ?actorResource rdfs:label ?actorName .
  FILTER(lang(?actorName) = "en")
}
```

   Una query di questo tipo svolge tre compiti:

   a. **Navigare la relazione**: Trova tutte le risorse collegate al film tramite la proprietà scelta (nell'esempio è `dbo:starring`, che significa "con la partecipazione di").

   b. **Estrarre l'etichetta leggibile**: Per ogni risorsa trovata (che è un URI), estrae il suo nome leggibile (`rdfs:label`).

c. **Filtrare per lingua**: Si assicura che il nome estratto sia in inglese (`FILTER(lang(...) = "en")`) per garantire la coerenza dei dati.

Questo schema viene ripetuto per tutte le altre proprietà ontologiche di DBpedia necessarie, come `dbo:director` per il regista, `dbo:musicComposer` per il compositore e `dbo:abstract` per il riassunto del film.

4. **Integrazione di dati da Wikipedia**: DBpedia è eccellente per i dati strutturati, ma alcune informazioni, come l'URL del poster di un film, non sono direttamente disponibili come proprietà del grafo. Tuttavia, DBpedia fornisce un collegamento diretto alla pagina Wikipedia corrispondente tramite la proprietà `foaf:isPrimaryTopicOf`.

   Il processo, quindi, prevede dei passaggi aggiuntivi:

   a. Tramite una query SPARQL, si ottiene l'URL della pagina Wikipedia del film.

   b. Si esegue uno `scraping` di quella pagina HTML.

   c. Utilizzando un selettore `XPath`, si individua l'elemento dell'immagine (`<img>`) all'interno dell'infobox del film per estrarre l'URL e scaricare il poster.

3.2.2 Descrizione dei file CSV generati

Il risultato finale del processo di acquisizione è un insieme di file CSV modulari, facili da caricare e da analizzare nelle fasi successive del sistema di raccomandazione. Ogni file rappresenta una specifica categoria di metadati e utilizza il `movieId` del dataset MovieLens come chiave di collegamento.

La maggior parte dei file adotta un formato a due colonne (`movieId` e `value`), utile a rappresentare le frequenti relazioni uno-a-molti (un film può avere più attori, più generi, ecc.).

Di seguito è riportata una descrizione dei file generati:

- **movie_genres.csv**: normalizza i generi dei film di MovieLens. A differenza del file originale `movies.csv`, dove i generi sono in un'unica stringa (es. "Action—Thriller"), questo file contiene una riga per ogni singola associazione `(movieId, genre)`. Viene generato dallo script `extract_genres.py`.

- **movie_actors.csv**: contiene le associazioni tra film e attori. Ogni riga rappresenta una coppia `(movieId, actor_name)`. Generato da `dbpedia_scraper.py`.

- **movie_composers.csv**: elenca i compositori delle colonne sonore per ogni film. La struttura è `(movieId, composer_name)`. Generato da `dbpedia_scraper.py`.

- **movie_directors.csv**: associa ogni film al suo regista (o registi). Ogni riga è una coppia `(movieId, director_name)`. Generato da `dbpedia_scraper.py`.

- **movie_producers.csv**: contiene le associazioni tra film e produttori. La struttura è `(movieId, producer_name)`. Generato da `dbpedia_scraper.py`.

- **movie_production_companies.csv**: elenca le case di produzione per ogni film. Ogni riga è una coppia `(movieId, company_name)`. Generato da `dbpedia_scraper.py`.
- **movie_subjects.csv**: associa i film a soggetti o temi specifici estratti da DBpedia (es. `http://dbpedia.org/resource/Category:Films_based_on_DC_Comics`). La struttura è `(movieId, subject_uri)`. Generato da `dbpedia_scraper.py`.
- **movie_writers.csv**: contiene gli sceneggiatori per ogni film. Ogni riga rappresenta una coppia `(movieId, writer_name)`. Generato da `dbpedia_scraper.py`.
- **movie_abstracts.csv**: memorizza il riassunto testuale (abstract) della trama di ogni film, estratto da DBpedia. La struttura è `(movieId, abstract_text)`. Generato da `dbpedia_scraper.py`.
- **existing_movies.csv**: funge da catalogo dei poster effettivamente scaricati, oltre a rappresentare l'elenco dei film che saranno utilizzati nell'interfaccia utente e dagli algoritmi di raccomandazione. Contiene `6725` dei `9742` film di MovieLens (ovvero il 69%). Le colonne sono `(movieID, movie_name)`. Viene generato dallo script `movies_by_posters_scraper.py`.

3.3 Preprocessing dei dati

Una volta ottenuta una ricca base di conoscenza sui film, il passo successivo è di combinare i dati soggettivi degli utenti con i metadati quantitativi precedentemente raccolti, così da ricavare un input per il nostro algoritmo di raccomandazione.

La figura 4 delinea l'architettura del sottosistema di data processing presente lato server, un package che raccoglie una serie di script Python (es. `vector_builder.py`, `build_similarity_matrix.py`, ...) responsabili delle operazioni di preparazione, trasformazione e analisi dei dati.

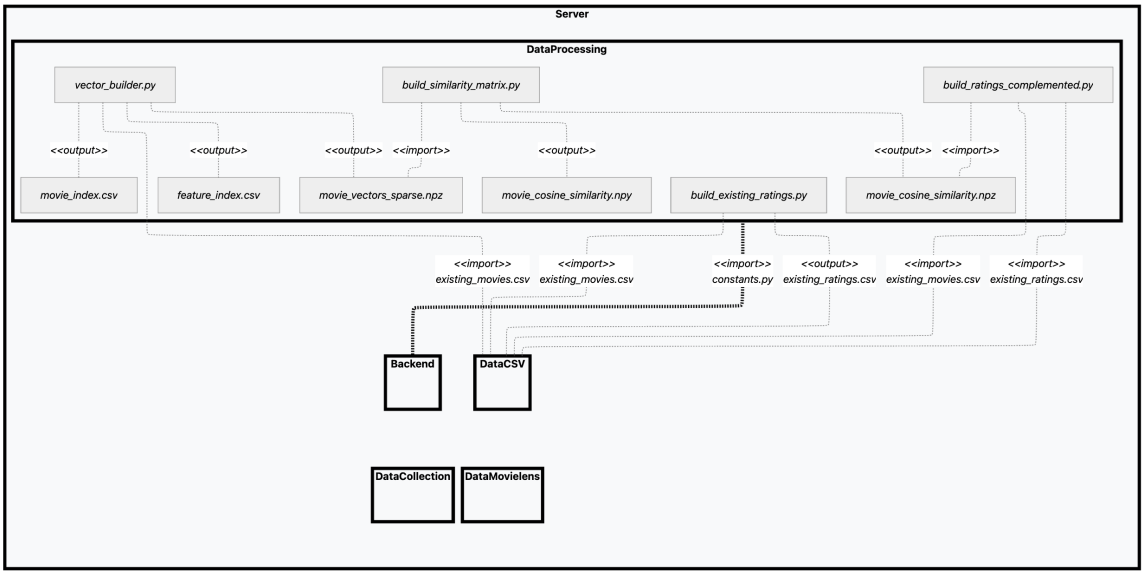


Figura 4: *Il package **DataProcessing** dedicato all'elaborazione dei dati*

Anche in questo caso, per ciascuno script sono mostrati i principali file di output generati (es. `movie_index.csv`, `feature_index.csv`, `movie_vectors_sparse.npz`, ...), nonché le dipendenze in input sotto forma di connessioni tra i moduli e i file utilizzati.

3.3.1 Vettorizzazione dei metadati

Per poter utilizzare i metadati qualitativi (come i nomi di attori, generi, ecc.) in un modello computazionale, è stato necessario trasformarli in una rappresentazione numerica. Questo processo, gestito dallo script `vector_builder.py`, ha l'obiettivo di costruire una *matrice sparsa film-feature* M, che codifica le proprietà di ogni film.

Le righe della matrice corrispondono univocamente ai film presenti nel file `existing_movies.csv`. Per mantenere un collegamento tracciabile, viene generato un file ausiliario, `movie_index.csv`, che mappa ogni `movie_id` all'indice di riga (da 0 a $N-1$, con N numero totale di film) corrispondente nella matrice.

Una "feature" non è semplicemente un valore (es. "Tom Hanks"), ma una coppia univoca `(categoria, valore)`, come ad esempio `('actor', 'Tom Hanks')` o `('genre', 'Sci-Fi')`. Questo approccio è fondamentale perché previene ambiguità: una persona potrebbe essere sia un attore che un regista, e questo schema permette di rappresentare i due ruoli come feature distinte. Lo script analizza tutti i file di metadati (es. `movie_actors.csv`, `movie_directors.csv`, etc.) per collezionare ogni singola coppia `(categoria, valore)` esistente, creando un vocabolario globale di tutte le possibili feature. Anche in questo caso, viene salvato un file di mappatura, `feature_index.csv`, che associa ogni feature unica a un indice di colonna (da 0 a $F-1$, con F numero totale di feature uniche).

Una volta definiti gli indici per righe e colonne, viene creata una matrice sparsa di dimensioni $N \times F$ (con N pari a `6725` ed F pari a `42044`):

$$M_{ij} = \begin{cases} 1 & \text{se il film } i \text{ possiede la feature } j \\ 0 & \text{altrimenti} \end{cases}$$

In questo modo, ogni film viene trasformato in un vettore binario con tante dimensioni quante sono le feature distinte trovate. Tale rappresentazione è la base per calcolare numericamente le somiglianze tra i film.

In Figura 5 è mostrato il grafico delle percentuali di feature che hanno un certo numero di film associati. Questi valori saranno di interesse per giustificare il filtro delle feature nell'algoritmo di raccomandazione.

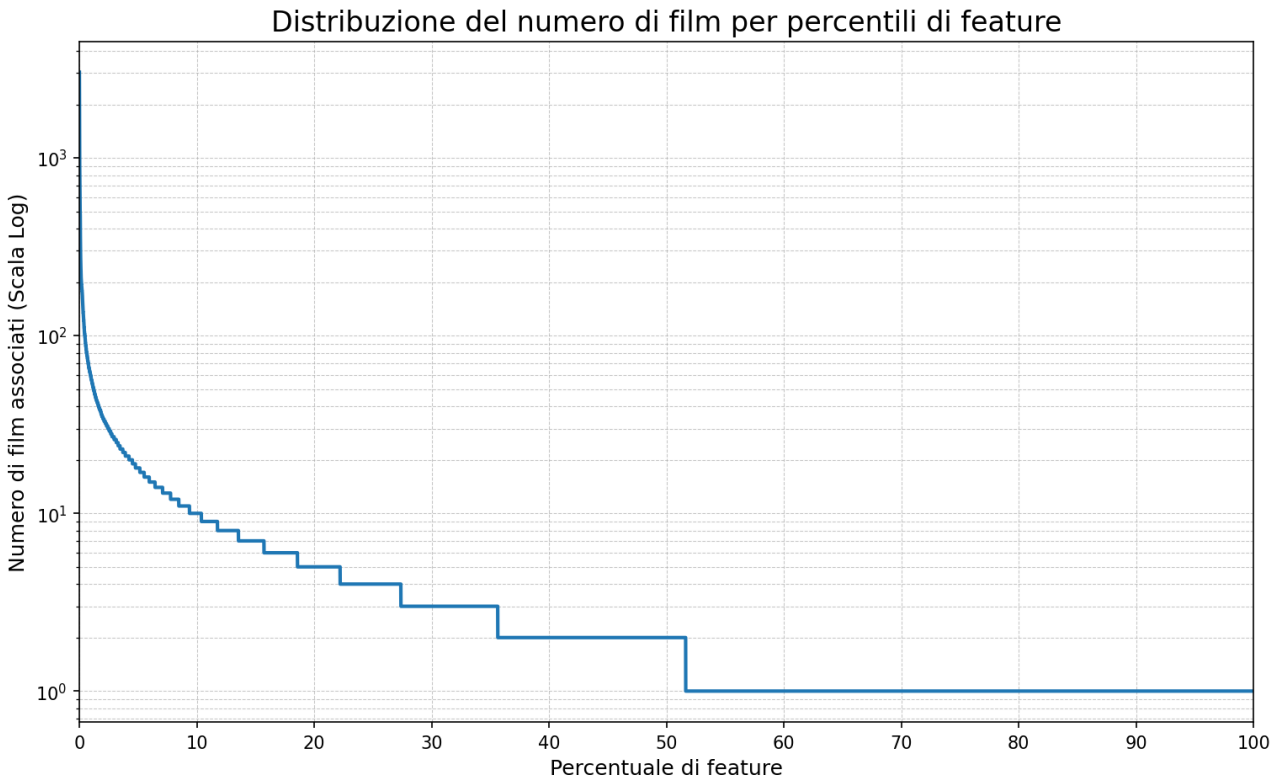


Figura 5: *Distribuzione della popolarità delle feature. L'asse X mostra i percentili delle feature. L'asse Y logaritmico mostra il numero di film associati a ciascuna percentuale feature.*

3.3.2 Misurare la similarità tra i film

Una volta che ogni film è rappresentato da un vettore di feature, il passo successivo è misurare quantitativamente quanto due film siano simili in base ai loro metadati. Per questo scopo, lo script `build_similarity_matrix.py` utilizza la *cosine similarity*, una metrica che calcola il coseno dell'angolo θ tra due vettori $\vec{A}$ e $\vec{B}$, che rappresentano due film:

$$\text{similarity}(\vec{A}, \vec{B}) = \cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\|\|\vec{B}\|} = \frac{\sum_{k=1}^{n} A_k B_k}{\sqrt{\sum_{k=1}^{n} A_k^2}\sqrt{\sum_{k=1}^{n} B_k^2}}$$

Il valore di similarità ottenuto è direttamente proporzionale alla "vicinanza" spaziale dei due vettori, cioè al numero di feature comuni. Inoltre, la cosine similarity è indipendente dalla magnitudine dei vettori perché il coseno è normalizzato rispetto alla loro norma L2 . In questo modo si evita che film con un numero molto elevato di feature (es. un blockbuster con molti attori e generi) risultino più simili ad altri solo perché i loro vettori sono meno sparsi.

Il calcolo viene eseguito dallo script `build_similarity_matrix.py`, che prende in input la matrice sparsa film-feature M e calcola la similarità coseno tra ogni coppia di film. Il risultato è una nuova matrice quadrata S, la *matrice di similarità*, di dimensioni $N \times N$, dove N è il numero totale di film. Ogni elemento S_{ij} di questa matrice contiene il punteggio di similarità tra il film i e il film j, compreso tra 0 e 1.

Come nota sulle ottimizzazioni adoperate, l'implementazione pratica si avvale delle funzio-

ni di Scikit-learn che permettono l'uso efficiente di matrici sparse, risparmiando memoria e velocizzando i calcoli algoritmici successivi.

```python
# Esempio semplificato di costruzione di una matrice di cosine similarity in
↪  build_similarity_matrix.py
from sklearn.metrics.pairwise import cosine_similarity
from scipy.sparse import save_npz

# X è la matrice sparsa film-feature (N_film x N_feature)
similarity_matrix = cosine_similarity(X, dense_output=False)

# Il risultato è una matrice (N_film x N_film) salvata in formato sparso
save_npz('movie_similarity_matrix.npz', similarity_matrix)
```

3.3.3 Predizione dei rating assenti

La matrice utente-film è intrinsecamente sparsa, siccome è normale che ogni utente abbia valutato solo una piccola frazione dei film disponibili. Tuttavia, per poter generare raccomandazioni personalizzate, è necessario stimare il gradimento di un utente per i film che non ha ancora visto. Questo processo di stima è gestito dallo script `build_ratings_complemented.py`.

L'approccio adottato è di tipo *content-based item-to-item*: per predire il rating $P_{u,i}$ che un utente u darebbe a un film non visto i, il sistema analizza i rating $r_{u,j}$ che lo stesso utente ha già assegnato ad altri film j. La predizione viene calcolata come una media dei rating noti, pesata con la **similarità basata sul contenuto** s_{ij} tra il film da predire i e ciascun film già valutato j. La similarità s_{ij} è esattamente quella calcolata nella fase precedente tramite la *cosine similarity* sui vettori di feature.

La formula per la predizione è la seguente:

$$P_{u,i} = \frac{\sum_{j \in \text{RatedItems}(u)} s_{ij} \cdot r_{u,j}}{\sum_{j \in \text{RatedItems}(u)} s_{ij}}$$

dove $\mathit{RatedItems}(u)$ è l'insieme dei film già valutati dall'utente u.

L'intuizione è che i voti dati a film molto simili a i (es. stesso genere, stesso regista) sono più influenti nel determinare il voto predetto per i. Ad esempio, se un utente ha valutato positivamente diversi film di fantascienza, il sistema predirrà un rating elevato anche per altri film di fantascienza che non ha ancora visto.

L'output di questo processo è un insieme di file CSV, uno per ciascuno dei 610 utenti (`ratings_complemented_user_<ID>.csv`), contenenti i rating predetti. Questi *rating complementati* consentono di trasformare un profilo di rating sparso di un utente in un profilo denso e completo.

3.4 Sistema di raccomandazione

Il sistema di raccomandazione implementato combina:

- Un'architettura *content-based* per la costruzione di un profilo di affinità quantitativo dell'utente e la selezione dei migliori candidati da raccomandare;

- Un approccio probabilistico basato su *MAB* per la scelta di una di lista di film da raccomandare che siano pertinenti, o nuovi e inaspettati.

La figura 6 a pagina 14 mostra l'architettura di questa implementazione. Il file `http_server.py` contiene sia il motore di raccomandazione sia le API REST per la comunicazione client-server, mentre i file `eval.py` e `movie_feature_analysis.py` sono adibiti alla valutazione del sistema. Tutti questi file interagiscono direttamente con il modulo di configurazione `constants.py`, da cui importano parametri e costanti condivise per la corretta esecuzione delle operazioni.

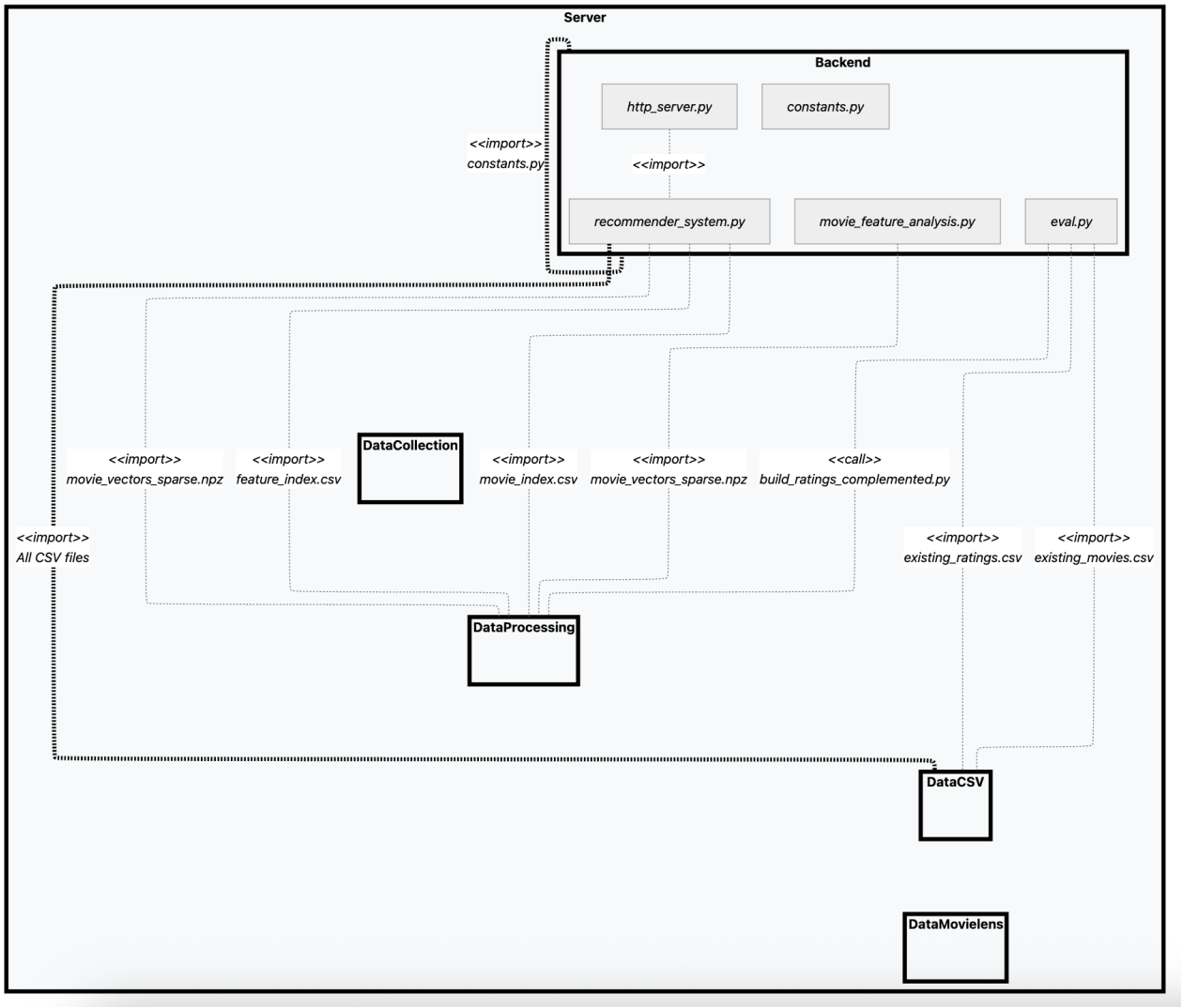


Figura 6: *Il package* ***Backend*** *incapsula il motore di raccomandazione, le API REST e il sistema di valutazione*

In particolare, le connessioni presenti denotano come gli output generati dalle precedenti fasi di acquisizione ed elaborazione in *DataProcessing* sono utilizzati in input per il motore di raccomandazione.

3.4.1 Profilazione utente content-based

La funzione `extract_user_preferences` costruisce un *profilo di affinità quantitativo* di un utente u verso le singole feature dei film (es. "Comedy", "Al Pacino", "1995 animated films"). Questo processo può essere formalizzato in tre passaggi algoritmici principali:

1. **Costruzione del vettore di rating Utente** ($\vec{r}_u$): Il primo passo consiste nel tradurre i rating dell'utente (quelli reali uniti ai rating predetti complementati) in un formato vettoriale compatibile con la matrice film-feature M (di dimensioni $N \times F$, dove N è il numero di film ed F il numero di feature).

   a. Viene creato un vettore denso $\vec{r}_u$ di dimensione N, inizializzato a zero.

   b. Per ogni film i valutato dall'utente u (sia con rating reali che complementati), il suo rating $r_{u,i}$ viene inserito nella posizione del vettore corrispondente all'indice del film i nella matrice M.

   Il risultato è un vettore $\vec{r}_u \in \mathbb{R}^N$ che rappresenta tutti i rating, reali e predetti, dell'utente, allineati con le righe della matrice M.

```python
# Parte della funzione extract_user_preferences in http_server.py

# 1a. Caricamento dei rating reali e complementati per l'utente
real_ratings, comp_ratings = load_user_ratings()
all_ratings = {**comp_ratings, **real_ratings}

# 1b. Costruzione del vettore denso di rating (ru)
# M e' il numero totale di film (N)
movie_ratings = np.zeros(M, dtype=float)
for movie_id, rating in all_ratings.items():

    idx = movie_index_df.loc[movie_index_df['movie_id'] == movie_id, 'matrix_id']
    idx = int(idx.iloc[0]) if not idx.empty else None

    if idx and 0 <= idx < M:
        movie_ratings[idx] = rating
```

2. **Calcolo del profilo di affinità** ($\vec{p}_u$): Il passaggio successivo è proiettare i rating dei film sulle feature, così da inferire le preferenze dell'utente per le feature. Questo viene realizzato tramite un'operazione di algebra lineare implementata nella funzione `compute_feature_means`.

   Sia M la matrice binaria film-feature, il profilo di affinità dell'utente $\vec{p}_u \in \mathbb{R}^F$ viene calcolato come:

$$\vec{p}_u = \frac{M^T \cdot \vec{r}_u}{M^T \cdot \vec{1}_N}$$

   dove la divisione è intesa come element-wise per calcolare una media dei rating per ciascuna feature. Analizziamo i passaggi:

- **Numeratore** ($M^T \cdot \vec{r}_u$): Il prodotto tra la trasposta della matrice delle feature e il vettore dei rating utente. Il j-esimo elemento del vettore risultato vettore risultante è la *somma di tutti i rating* che l'utente u ha dato ai film che possiedono la feature j.

- **Denominatore** ($M^T \cdot \vec{1}_N$): Il prodotto tra M^T e un vettore unitario di dimensione N. Il j-esimo elemento di questo vettore è semplicemente il *conteggio totale* dei film che possiedono la feature j.

Il risultato, $\vec{p}_u$, è un vettore di dimensione F dove ogni componente $(\vec{p}_u)_j$ rappresenta il **rating medio** che l'utente u ha assegnato o assegnerebbe alla feature j. Questo vettore è a tutti gli effetti il "profilo di affinità" dell'utente rispetto a ciascuna feature esistente.

```python
    # Funzione compute_feature_means in http_server.py

    def compute_feature_means(ratings, min_support: int):
        # 'ratings' e' il vettore ru (N x 1)
        # 'movie_features_matrix' e' la matrice M (N x F)

        # Numeratore: M^T * ru -> Somma dei rating per feature (F x 1)
        sum_per_feature = movie_features_matrix.T.dot(ratings)

        # Denominatore: M^T * 1_N -> Conteggio film per feature (F x 1)
        count_per_feature = movie_features_matrix.T.dot(np.ones(M, dtype=float))

        # Calcolo della media (divisione element-wise)
        means = np.divide(
            sum_per_feature,
            count_per_feature,
            out=np.zeros_like(sum_per_feature),
            where=(count_per_feature != 0)
        )

        # Filtro di supporto minimo
        mask = count_per_feature >= min_support

        return means * mask # Restituisce il vettore pu (F x 1)
```

3. **Estrazione delle "Top Features"**: il profilo di affinità delle feature $\vec{p}_u$ viene ordinato in base al valore di rating medio precedentemente ottenuto. Inoltre, viene applicato un filtro di *supporto minimo* per escludere feature con poche occorrenze.

   Questa lista di "top features" costituisce l'insieme dei film che hanno almeno una delle suddette feature e che fungeranno da input per la fase successiva di generazione delle raccomandazioni. Questo approccio risolve il problema del *cold start* per i nuovi film: un film può essere raccomandato non appena i suoi metadati sono noti, senza bisogno di essere stato valutato da altri utenti.

```python
    # Funzione extract_user_top_features in http_server.py

    def extract_user_top_features(feature_means, top_features: int):
    # 'feature_means' e' il vettore di affinita' pu

    if (feature_means.sum() > 0.0):
        # Ordinamento decrescente degli indici delle feature in base al rating
        ↪ medio
        top_indices = np.argsort(-feature_means)[:top_features]

        # Costruzione della lista delle top features con i loro metadati
        for idx in top_indices:
            row = feature_index_df.iloc[idx] # Recupera info dal CSV delle feature
            feature_rating_value = feature_means[idx]

            top_features_list.append({
                "id": int(row["feature_id"]),
                "category": row["category"],
                "name": row["feature"],
                "rating": float(feature_rating_value)
            })
```

3.4.2 MAB: Softmax Exploration

Una volta costruito il profilo delle preferenze dell'utente e identificate le "top features", il sistema deve affrontare il problema di come selezionare i film specifici da raccomandare. Un approccio deterministico, basato sulla pura *exploitation*, si limiterebbe a mostrare i film non visti con il rating predetto più alto. Sebbene pertinenti, tali raccomandazioni porterebbero rapidamente a suggerimenti monotoni.

Per bilanciare *exploration* ed *exploitation*, il sistema implementa una strategia basata sul *MAB*. Ogni "top feature" del profilo utente viene trattata come un "bandit" (una slot machine) e i film associati a quella feature sono le "leve" da tirare. La strategia specifica adottata è la *Softmax exploration*, implementata nella funzione `mab_softmax_predictions`.

Questo algoritmo non sceglie deterministicamente il film "migliore", ma tratta i rating predetti come dei reward per convertire l'insieme di film candidati in una distribuzione di probabilità. Pertanto, la probabilità di raccomandare un film i è calcolata tramite la funzione softmax:

$$P(\text{film } i) = \frac{e^{r_i/\tau}}{\sum_{j \in \text{Candidati}} e^{r_j/\tau}}$$

dove r_i è il rating predetto per il film i (la ricompensa attesa), e τ è il parametro della *temperatura*, che modula quanto vogliamo una raccomandazione più o meno esplorativa:

- Un $\tau \to 0$ rende l'algoritmo (*greedy*), concentrando quasi tutta la probabilità sulla leva con la ricompensa più alta (pura exploitation).

- Un τ più alto rende la distribuzione più uniforme, aumentando la probabilità di selezionare anche opzioni sub-ottimali (maggiore exploration).

Infine, il sistema non restituisce i "top-k" film ordinati per punteggio, ma *campiona* k film da questa distribuzione di probabilità. Come mostrato in figura, questo campionamento viene eseguito tramite la funzione `np.random.choice`, passando le probabilità calcolate come pesi. Questa scelta è fondamentale: garantisce che le raccomandazioni siano pertinenti (i film con rating alto sono più probabili) ma non scontate.

```python
def mab_softmax_predictions(temperature: float, k: int):
    # Fallback a un valore di default se k non e' valido
    if isinstance(k, int) and k < 0:
        k = MOVIE_RECOMMENDATIONS

    predictions = []
    min_k = k

    # Itera su ogni "top feature" del profilo utente
    for f in top_features_list:
        movies = f.get("movies", [])

        if not movies:
            continue

        # 1. Filtro: considera solo i film non ancora visti dall'utente
        # La tupla 'm' e' strutturata come (movieId, rating, seen_bool)
        movies = [m for m in movies if not m[2]]
        if not movies:
            continue

        # Se k=0, imposta il numero di raccomandazioni pari a tutti i film candidati
        if k == 0:
            min_k = len(movies)

        # 2. Estrazione dei rating (rewards) per i film candidati
        ratings = np.array([m[1] for m in movies], dtype=float)

        # 3. Calcolo della distribuzione di probabilita' tramite Softmax
        # La temperatura (tau) bilancia exploration e exploitation
        probs = np.exp(ratings / temperature)
        probs /= np.sum(probs)

        # 4. Campionamento probabilistico di k film
        # La selezione avviene in base alla distribuzione 'probs'
        sample_size = min(min_k, len(movies))
        chosen_idx = np.random.choice(
            len(movies),
            size=sample_size,
            replace=False,
            p=probs
        )

        # 5. Costruzione dell'output JSON con i film campionati
        predictions.append({
```

```
            "feature_id": f["id"],
            "category": f["category"],
            "feature_name": f["name"],
            "feature_rating": f["rating"],
            "movies": {
                movies[idx][0]: {
                    "movie_rating": movies[idx][1],
                    "seen": movies[idx][2],
                    "softmax_prob": probs[idx]
                }
            for idx in chosen_idx}
        })

    return predictions
```

3.5 Il sistema client-server per la comunicazione con l'utente

Nel contesto del sistema di raccomandazione sviluppato, la comunicazione efficace e bidirezionale tra utente e motore di raccomandazione è garantita da una solida architettura client-server. Tale architettura consente di separare la presentazione delle informazioni (client) dalle logiche di gestione dei dati e di raccomandazione (server). Tramite l'esposizione di API REST e una chiara divisione in moduli, il sistema assicura scalabilità, manutenibilità e sicurezza, oltre a una gestione ottimale delle richieste e delle risposte tra le due componenti.

Il client è stato progettato seguendo il paradigma Model-View-Controller (MVC), che garantisce una separazione chiara tra logica di presentazione, gestione dei dati e controllo dei flussi applicativi. Questa organizzazione modulare migliora la manutenibilità e assicura una user experience coerente e scalabile. Nei paragrafi seguenti verranno illustrati nel dettaglio i componenti principali, le modalità di interazione, e le dipendenze software che sottendono la comunicazione in tutto il processo di raccomandazione.

3.5.1 Comunicazione tramite API REST

Il package **Services** aggrega diversi moduli fondamentali che svolgono ruoli specifici a supporto dell'interfaccia utente per la comunicazione tramite API REST con il server, nonché le logiche di routing e validazione.

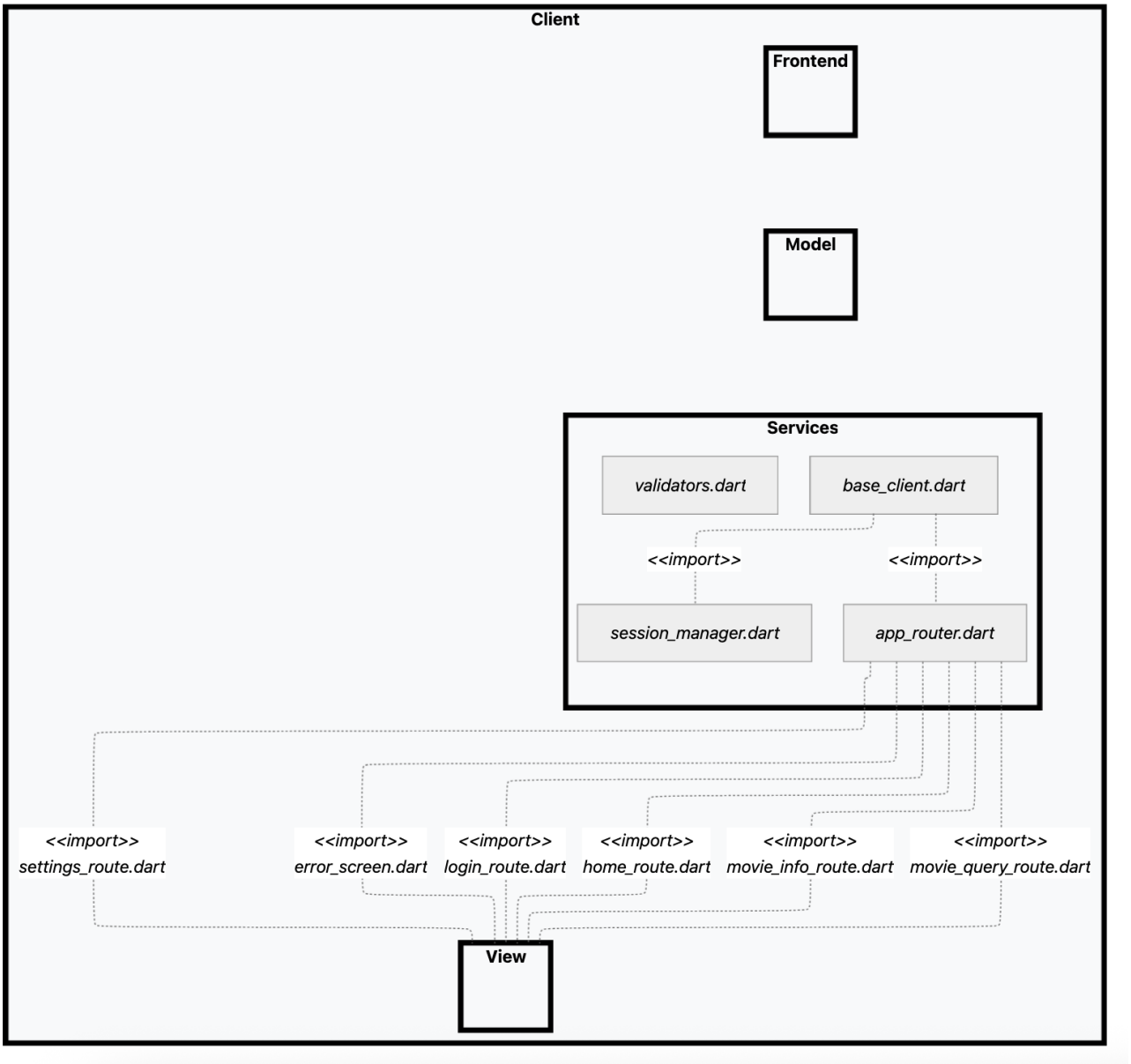


Figura 7: *Il package* ***Services*** *adibito alla comunicazione con il server*

Tale package comprende quattro file principali:

- *`app_router.dart`*: contiene la classe **AppRouter**, che gestisce la navigazione dell'applicazione utilizzando il pacchetto *"go_router"*. Il diagramma evidenzia, tramite delle frecce tratteggiate etichettate con << *import* >>, i file del package **View** che vengono utilizzati per la gestione delle varie schermate (es. `settings_route.dart`, `home_route.dart`, ...).

- *`base_client.dart`*: contiene la classe **BaseClient** che implementa un client HTTP per effettuare richieste API verso il server Python, utilizzando il pacchetto *"Dio"* per la gestione delle richieste e delle risposte.

- *session_manager.dart*: contiene la classe **SessionManager**, utilizzata per gestire lo stato di autenticazione dell'utente nell'applicazione, implementando metodi per il login e il logout tramite l'uso della libreria *"shared_preferences"* per la persistenza dei dati.

- *validators.dart*: include la classe **Validators**, la quale fornisce metodi per la validazione di email e indirizzi, utilizzando regex per verificare i pattern.

L'implementazione delle API REST costituisce il principale canale di interazione tra il client e i servizi back-end.

In particolare, la classe *BaseClient* lato client implementa dei metodi per la comunicazione tramite protocollo HTTP con la classe *RecSys_RequestHandler* lato server. Tutte le risposte sono fornite in formato JSON, e sono successivamente elaborate lato client per ottenere le informazioni richieste.

- Endpoint `/get-users`: restituisce la lista degli identificativi univoci degli utenti presenti nel sistema. Questa API è utilizzata in fase di login per l'avvio di una sessione di raccomandazioni personalizzate.

  – API lato client: `Future<dynamic> getUsers()`

  – API lato server: `urlparse(self.path).path.endswith('/get-users')`

- Endpoint `/get-params`: fornisce i parametri globali attualmente configurati nel sistema di raccomandazione. In particolare, restituisce un oggetto JSON contenente i valori di `minSupport`, `movieRecommendations` e `topFeatures` [2]. Viene utilizzata nella schermata di impostazioni dell'applicativo per ripristinare i valori correnti.

  – API lato client: `Future<dynamic> getParams()`

  – API lato server: `urlparse(self.path).path.endswith('/get-params')`

- Endpoint `/get-recommendations`: genera un insieme di raccomandazioni basate su un modello *MAB* con politica *softmax*, utilizzando la funzione `mab_softmax_predictions(temperature:float, k:int)`. Questa API rappresenta il nucleo funzionale del sistema di raccomandazione, e viene utilizzata nella schermata principale dell'applicativo.

  – API lato client: `Future<dynamic> getMovieRecommendations()`

  – API lato server: `urlparse(self.path).path.endswith('/get-recommendations')`

- Endpoint `/get-movies`: restituisce la lista di identificativi di film valutati da un determinato utente oppure associati ad una specifica feature. L'endpoint viene utlizzato per filtrare e ordinare i film e mostrarli nella schermata dedicata alle query sul dataset.

  – API lato client: `Future<dynamic> getMoviesFromFeature(required String`

[2] Si veda la documentazione interna per maggiori dettagli su questi parametri.

`featureId, String? order)`

  - API lato client: `Future<dynamic> getMoviesFromRatings(required String userId, String? order)`

  - API lato server: `urlparse(self.path).path.endswith('/get-movies')`

- Endpoint `/download-movie-poster`: consente di scaricare la copertina associata a un film specifico. La richiesta utilizza il parametro `id` per identificare il film desiderato. Il server ricerca il file corrispondente nella directory dei poster e, se disponibile, lo restituisce in formato `image/jpeg`.

  - API lato client: `Future<dynamic> downloadMoviePoster(required String idMovie)`

  - API lato server: `urlparse(self.path).path.endswith('/download-movie-poster')`

- Endpoint `/get-movie-info`: fornisce informazioni dettagliate su un film specifico, identificato dal parametro `id`. L'API aggrega i dati provenienti dal package **DataCSV** (es. titoli, descrizioni, ...), restituendo un oggetto JSON strutturato. Questa API è utilizzata nella schermata di visualizzazione dei dettagli di un singolo film.

  - API lato client: `Future<dynamic> getMovieInfo(required String idMovie, String? type)`

  - API lato server: `urlparse(self.path).path.endswith('/get-movie-info')`

- Endpoint `/login-user`: gestisce la procedura di autenticazione di un utente nel sistema di raccomandazione. La richiesta deve contenere un corpo in formato JSON con il campo `userId`. In caso di validità del payload, il server esegue la funzione `extract_user_preferences()`, che inizializza il profilo utente in base ai parametri di raccomandazione correnti.

  - API lato client: `Future<dynamic> loginUser(required String userId)`

  - API lato server: `urlparse(self.path).path.endswith('/login-user')`

- Endpoint `/update-params`: consente di aggiornare in modo dinamico, cioè senza necessità di riavvio del server, i parametri operativi del sistema di raccomandazione.

  - API lato client: `Future<dynamic> updateParams(int? minSupport, int? movieRecommendations, int? topFeatures)`

  - API lato server: `urlparse(self.path).path.endswith('/update-params')`

3.5.2 Rappresentazione dei dati

Il package **Model** si occupa della rappresentazione strutturata dei dati all'interno dell'applicazione client, grazie alle classi di modello per gestire, serializzare e trasferire le informazioni tra le diverse componenti del sistema.

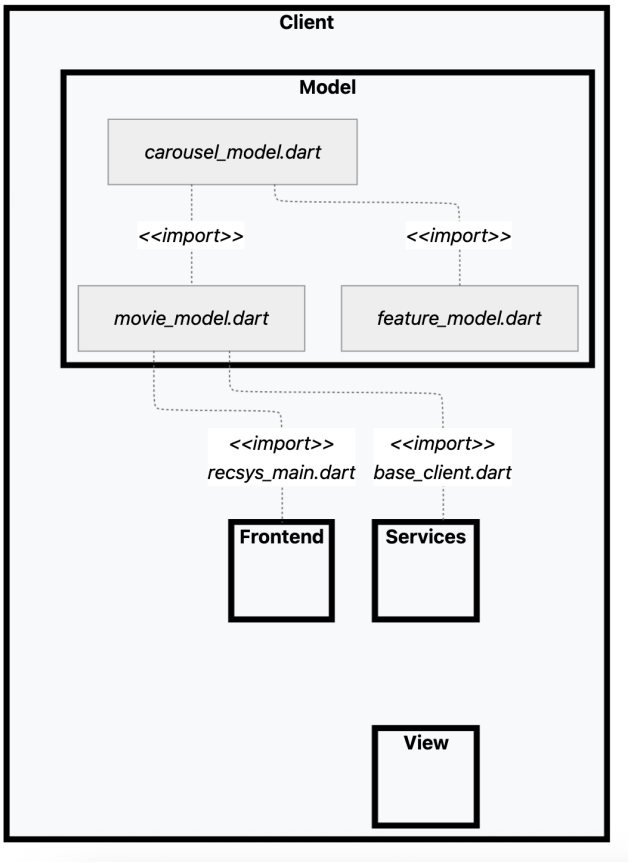


Figura 8: *Il package **Model** descrive la rappresentazione dei dati provenienti dal server*

L'architettura illustrata nel diagramma evidenzia le relazioni di dipendenza tra i modelli stessi e gli altri package logici del client, garantendo un'organizzazione efficiente, estendibile e facilmente manutenibile.

Tale package comprende tre file principali:

- *carousel_model.dart*: la classe **Carousel** rappresenta un carosello di film raccomandati, associato a una particolare caratteristica (feature) del sistema di raccomandazione. Viene utilizzata per organizzare e visualizzare le raccomandazioni di film in modo strutturato e intuitivo per l'utente. Ad ogni istanza di **Carousel** è associata una singola **Feature** e una lista di **Movie**, che corrispondono ai film raccomandati dal sistema.

- *feature_model.dart*: la classe **Feature** rappresenta una caratteristica di un film. Ognuna di esse ha un identificativo univoco, una categoria (es. genres, directors, ...), il nome che,

appunto, la descrive e un rating medio basata sulle valutazioni dell'utente reali e predette dal sistema.

- *movie_model.dart*: la classe **Movie** rappresenta un film, con i suoi metadati principali (es. id, titolo, descrizione, cast, ...) e viene utilizzata per memorizzare e gestire le informazioni sui film presenti nel database dell'applicazione di raccomandazione.

3.5.3 Gestione dell'interfaccia utente

Il package **View** rappresenta il cuore dell'applicazione lato client. Tale package racchiude tutti i file DART che implementano sia le diverse schermate sia i widget personalizzati.

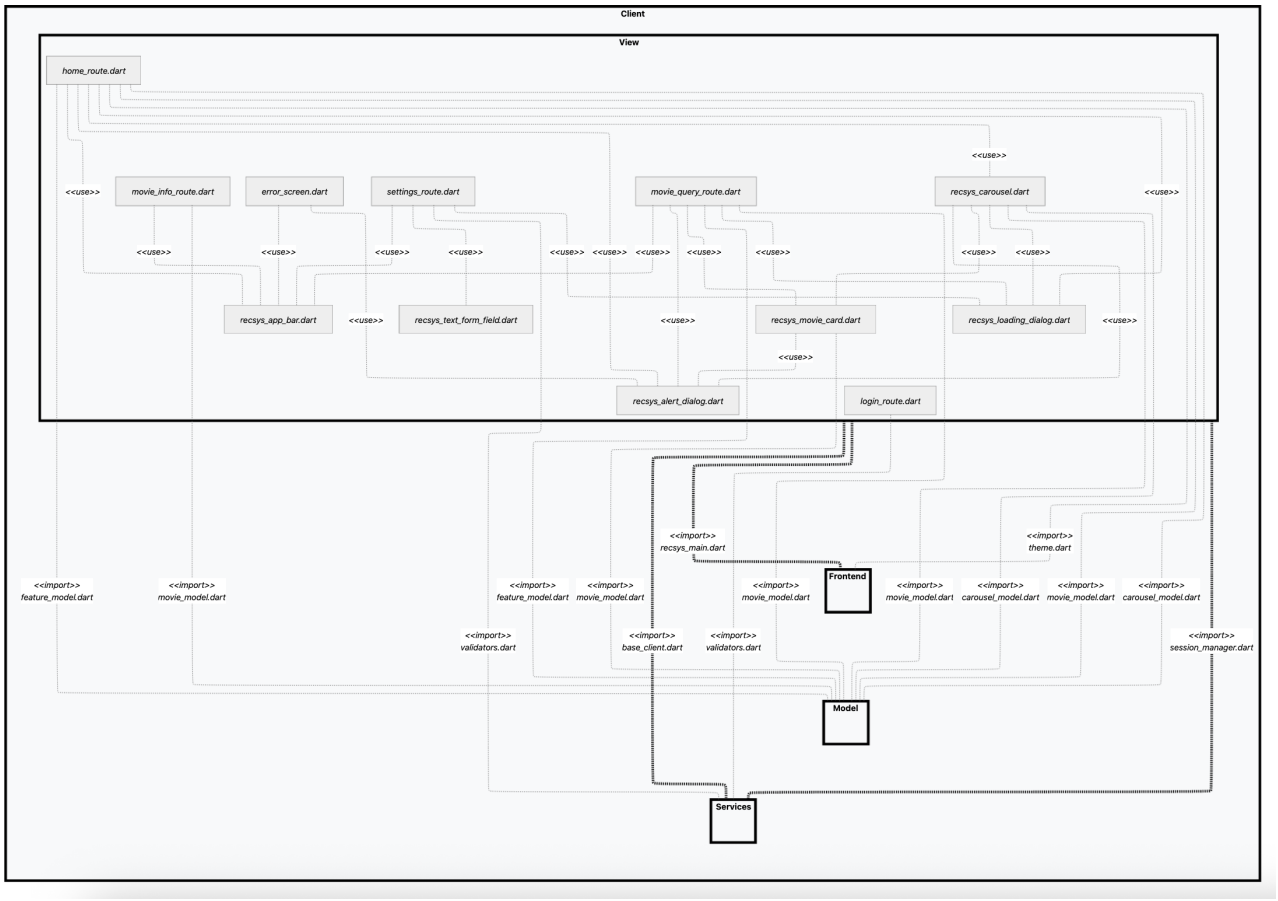


Figura 9: *Il package **View** nel quale sono implementate le schermate dell'applicativo client*

Le relazioni tra le componenti sono espresse tramite due tipologie di etichette: << *use* >> indica che una schermata fa uso di uno o più widget personalizzati; << *import* >> segnala che una schermata importa dati o funzionalità da altri package. Questo evidenzia una netta separazione dei livelli, in totale accordo con il paradigma di progettazione MVC.

Ogni schermata offre un'esperienza utente chiara e intuitiva, grazie all'integrazione efficiente di widget riutilizzabili e di una schermata di error handling:

- `routes/home_route.dart`: rappresenta la schermata principale dell'applicazione dove l'utente può visualizzare le principali raccomandazioni che il sistema ha generato, in base ai metadati dei film presenti nel database e alle preferenze dell'utente stesso.

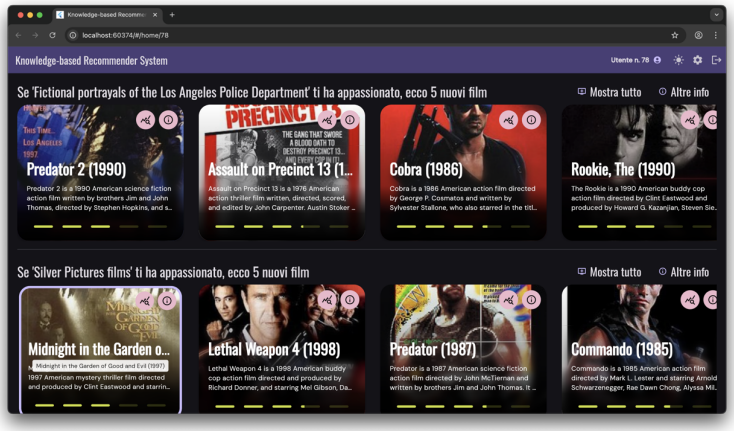


Figura 10: *La schermata principale offerta da* ***HomeRoute***

Inoltre, da questa schermata l'utente può accedere alle altre funzionalità dell'applicazione, tra le quali annoveriamo:

– La visualizzazione delle statistiche di un carosello (cliccando sul pulsante *Altre info* in testa al carosello), che mostra una serie di informazioni aggiuntive specifiche della feature raccomandata.

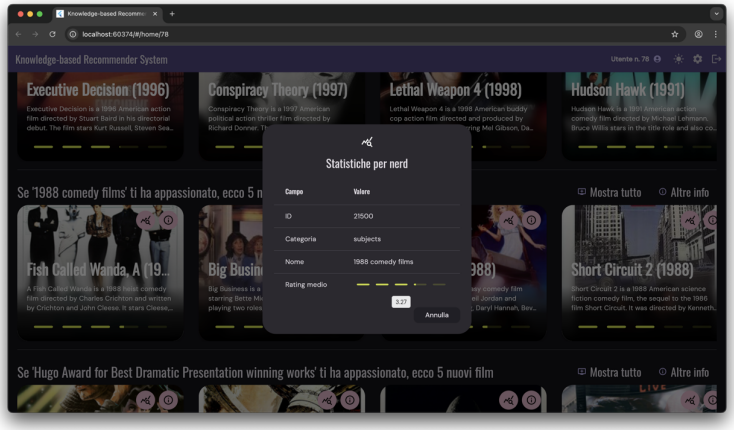


Figura 11: *Le statistiche di una feature in* ***HomeRoute***

– Il caricamento di tutti i film legati ad una delle feature raccomandate (cliccando sul pulsante *Mostra tutto* in testa al carosello), che mostra la schermata **MovieQueryRoute**.

- *routes/login_route.dart*: include la schermata di login dell'applicazione dove l'utente può inserire il proprio identificativo per accedere al sistema e inizializzare la piattaforma di raccomandazione in base ai rating che ha fornito. L'utente, una volta effettuato l'accesso, viene reindirizzato alla schermata principale dell'applicazione.

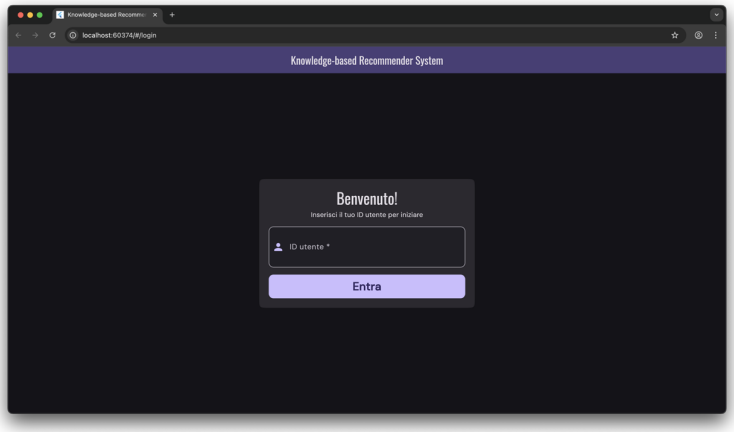


Figura 12: *La schermata di login offerta da* ***LoginRoute***

- *routes/movie_info_route.dart*: gestisce la schermata dei dettagli di un film dove l'utente può visualizzare tutti i metadati (e.g. titolo, descrizione, cast, ...) del film selezionato [3].

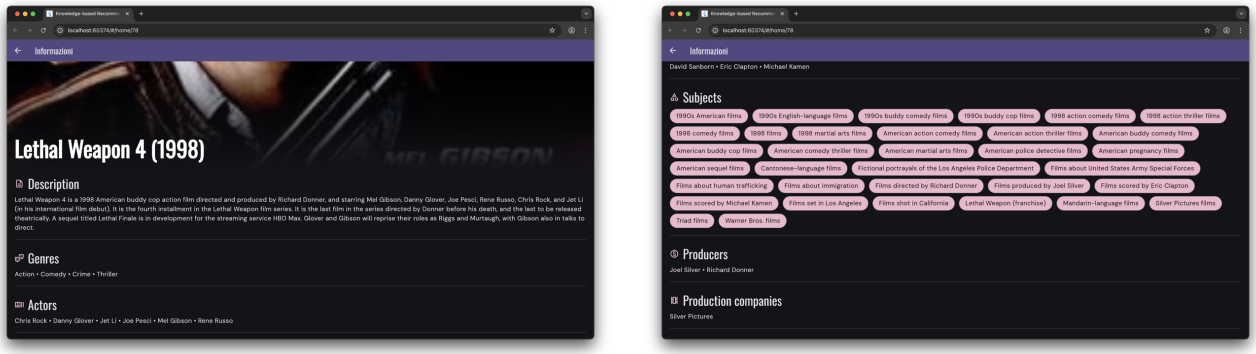
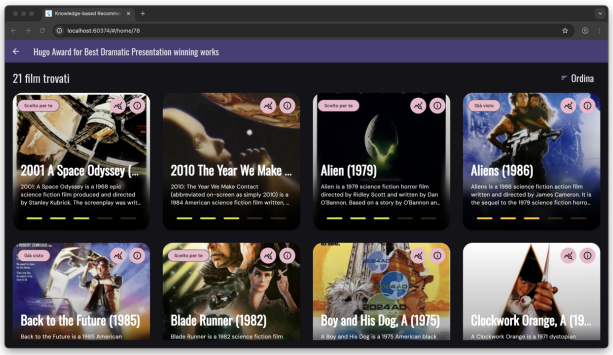


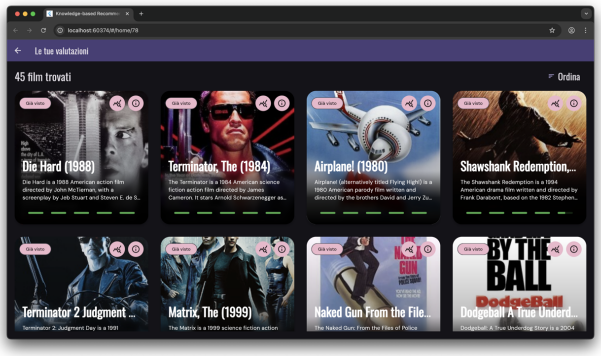
Figura 13: *La schermata dei dettagli di un film offerta da* ***MovieInfoRoute***

[3] Questa schermata può essere raggiunta cliccando sul pulsante *Info* di una **RecSysMovieCard**

- *routes/movie_query_route.dart*: in questa schermata, sono mostrati all'utente tutti i film restituiti da una query al database del sistema in una griglia scrollabile.



(a) *L'utente clicca Mostra tutto su uno dei caroselli della* ***HomeRoute***



(b) *L'utente clicca il pulsante Le tue valutazioni nella AppBar della* ***HomeRoute***

Figura 14: *La schermata di query al database offerta da* ***MovieQueryRoute***

Inoltre, da questa schermata l'utente può:

– Visualizzare i dettagli tecnici della raccomandazione (cliccando sul pulsante *NerdStats* di una card) [4].

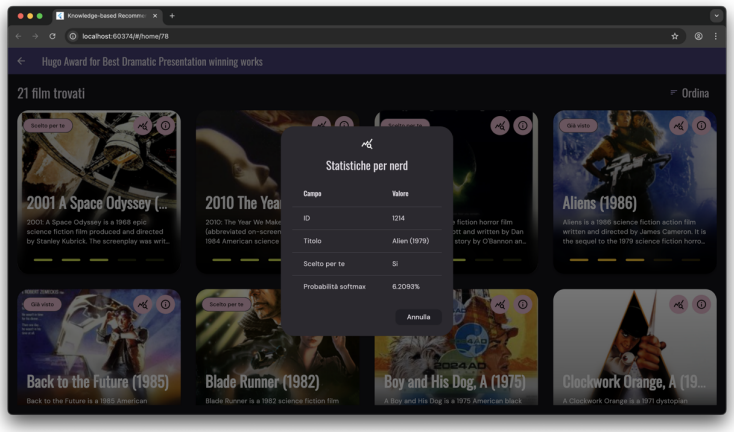


Figura 15: *Dettagli tecnici di una raccomandazione dal widget* ***RecSysMovieCard*** *in* ***MovieQueryRoute***

[4] In particolare, quest'operazione è possibile anche in altre schermate che contengono queste card di film, grazie al riutilizzo del widget **RecSysMovieCard**.

– Modificare l'ordinamento dei risultati della query (cliccando sul pulsante *Ordina* in testa alla schermata).

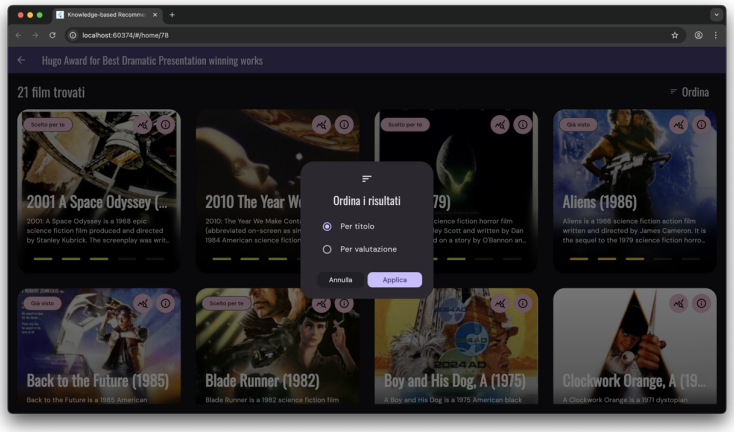


Figura 16: *Ordinamento dei risultati della query in* ***MovieQueryRoute***

- `routes/settings_route.dart`: mostra una una schermata dove l'utente può modificare i parametri del sistema di raccomandazione.

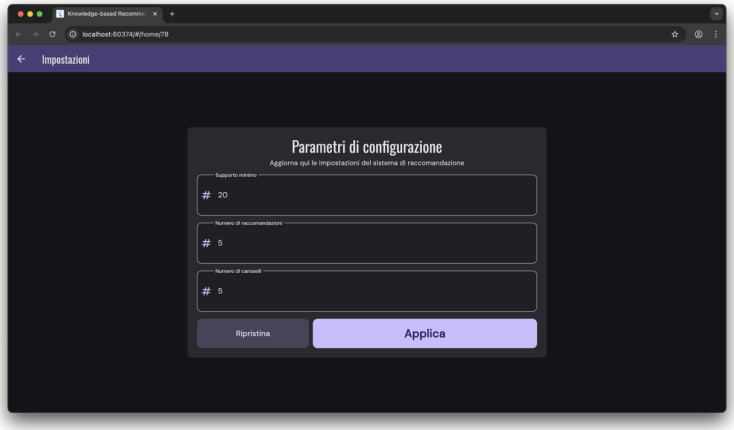


Figura 17: *La schermata delle impostazioni offerta da* ***SettingsRoute***

- *screens/error_screen.dart*: visualizza un messaggio di errore generico quando si verifica un problema durante la navigazione o il caricamento di una schermata dell'applicazione.

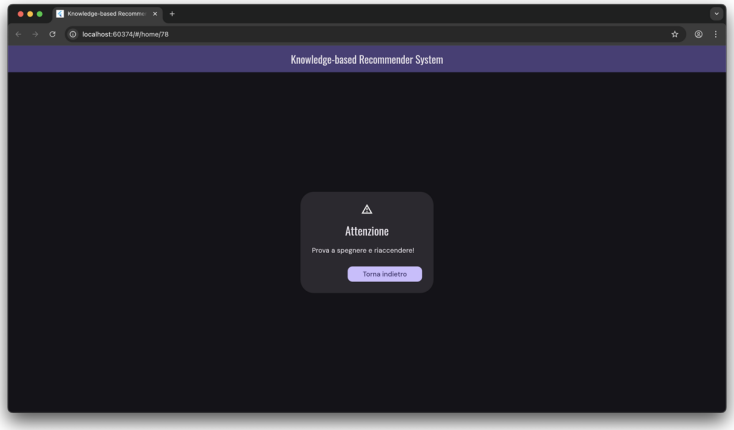


Figura 18: *Una schermata di errore offerta da* ***Error-Screen***

4 Spiegabilità delle raccomandazioni

L'interfaccia utente è stata progettata con particolare attenzione agli aspetti di spiegabilità (explainability) delle raccomandazioni fornite dal sistema, in modo che possa fornire all'utente motivazioni intellegibili per le raccomandazioni suggerite, rafforzando la fiducia e la comprensione del comportamento algoritmico.

4.1 Scelte di progettazione

Come mostrato nella Figura 10 a pagina 25, la parte superiore di ciascun carosello mostra chiaramente il titolo della feature raccomandata esplicitando quale caratteristica è stata utilizzata per la selezione dei film proposti. Infatti, l'approccio content-based facilita per definizione la produzione di spiegazioni "naturali" in quanto l'algoritmo di raccomandazione si basa sulla somiglianza fra item e sulle preferenze manifestate dall'utente su elementi analoghi.

Sulle card dei singoli film, è sempre ben visibile il rating (reale o predetto), il quale riflette il profilo di ciascun utente distinguendo tra utenti più critici (con voti generalmente bassi) e utenti enthusiast (con voti tendenzialmente elevati). Questa scelta di progettazione offre nell'interfaccia un indicatore misurabile della rilevanza di ciascun suggerimento rispetto alle preferenze individuali, e ne permette una comprensione oggettiva.

Ulteriori dettagli tecnici sulle raccomandazioni sono accessibili tramite il pulsante "Altre info", dove l'utente può consultare anche la probabilità di selezione calcolata via softmax, ottenendo così una visione ancora più approfondita e trasparente del processo decisionale.

4.2 Metriche utilizzate

Per articolare le motivazioni sottostanti al sistema di raccomandazione, risulta fondamentale introdurre un insieme di metriche complementari che catturino aspetti distinti dell'esperienza utente e dell'utilità percepita del sistema.

Le metriche R/N/S/D permettono di esplicitare gli obiettivi operativi del sistema di raccomandazione. Ciascuna metrica risponde a una specifica dimensione della relazione tra utente e contenuto, e la loro combinazione offre una visione complessiva delle scelte algoritmiche operate.

- ***Relevance***: misura il grado di allineamento tra gli item proposti e le preferenze dell'utente. Nell'interfaccia utente la **Relevance** è evidenziata attraverso il rating predetto o reale degli item, che permette di comprendere se il sistema è stato in grado di identificare contenuti potenzialmente graditi.

- ***Novelty***: valorizza la capacità del sistema di ampliare l'orizzonte conoscitivo dell'utente, proponendo contenuti non ovvi ma comunque pertinenti. In altre parole, misura la capacità del sistema di suggerire item all'utente che non avrebbe scoperto autonomamente.

- ***Serendipity***: cattura l'effetto sorpresa positivo, ovvero la capacità del sistema di proporre item inaspettati che si rivelano graditi nonostante si discostino dai pattern usuali. In pra-

tica, a differenza della **Novelty**, rappresenta l'obiettivo di un sistema di raccomandazione di stimolare curiosità e apertura verso contenuti non riconducibili al profilo utente.

- ***Diversity***: valuta la ridondanza tematica di tutto l'insieme di raccomandazioni. Infatti, un insieme di suggerimenti si ritiene efficace se garantisce una certa varietà di features e categorie.

4.3 Casi utente con screenshot

Nei casi utente che seguono, grazie all'utilizzo congiunto delle metriche R/N/S/D e degli screenshot, si mettono in evidenza chiaramente le logiche decisionali del sistema favorendo la comprensibilità delle decisioni.

4.3.1 Utente n. 3

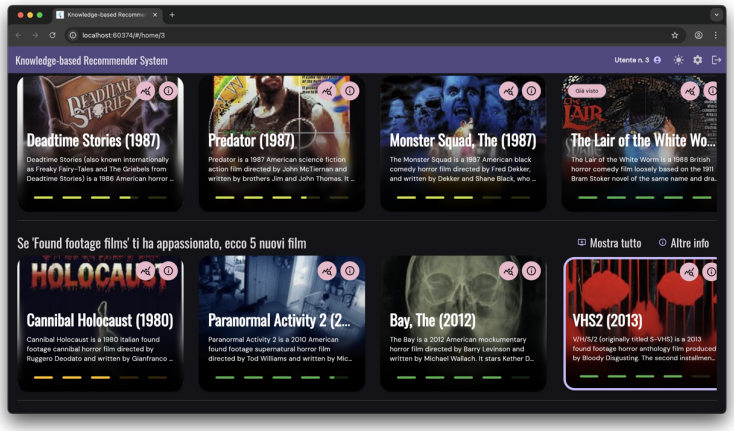


Figura 19: *Caso utente n. 3 - Esempio 1*

Lo screenshot rivela un profilo coerente e ben focalizzato sul genere *science fiction horror*, con una preferenza marcata per produzioni degli *anni '80*.

Nella feature più rappresentativa (cioè *1980s science fiction horror films*), l'utente ha già espresso una valutazione con *rating* = 5.0 per il film *Saturn 3 (ID: 2851)*. Questo rafforza l'associazione tra gli anni '80 e il sottogenere horror fantascientifico.

La presenza della feature *2010s science fiction horror films* suggerisce un tentativo del sistema di espandere i gusti dell'utente verso produzioni più recenti. Nonostante la coerenza di genere, i film proposti mostrano una maggiore dispersione dei rating (da 2.77 a 4.23). Questo pattern evidenzia una diversificazione che resta semanticamente compatibile con le preferenze esistenti, ma introduce nuove epoche e registi.

Infine, l'inclusione della feature *Found footage films* rappresenta il principale vettore di **Serendipity**. Sebbene si tratti di un sottogenere affine all'horror ma con una struttura narrativa sperimentale (e potenzialmente distante dai gusti consolidati), i titoli raccomandati presentano

valori di rating molto alti (fino a 4.43). Ciò indica che il sistema riconosce in questa direzione un potenziale di gradimento inaspettato ma promettente.

Un caso molto particolare è proprio il film *Saturn 3 (ID: 2851)*.

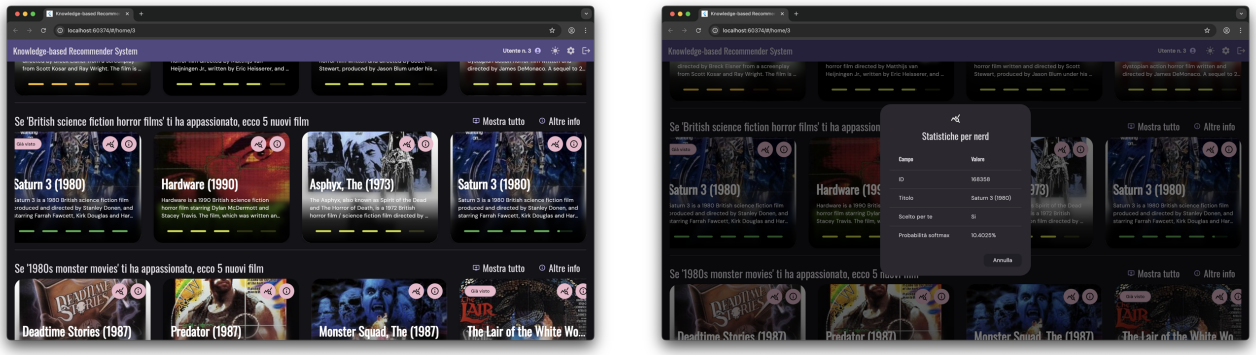


Figura 20: *Caso utente n. 3 - Esempio 2*

Come detto prima, l'utente ha già visto e valutato questo film con *rating* = 5.0. Tuttavia, viene comunque raccomandato con un rating predetto molto alto pari a 4.27 perché esiste una copia con caratteristiche estremamente simili (ID: 168358) nel dataset, evidenziando come il sistema riesca a generalizzare le preferenze su contenuti simili.

4.3.2 Utente n. 5

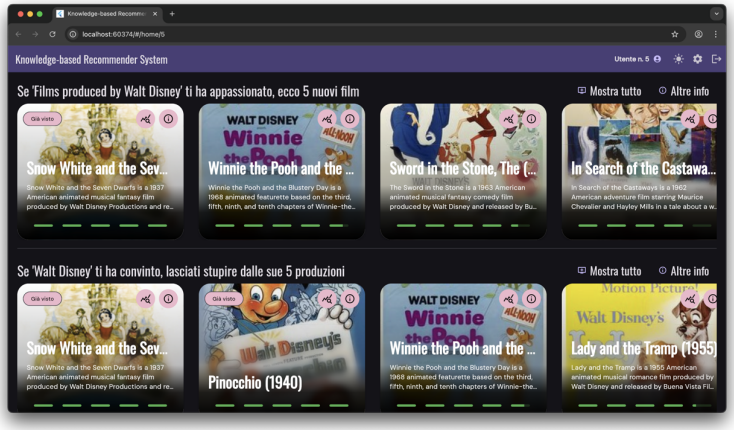


Figura 21: *Caso utente n. 5 - Esempio 1*

Da questo screenshot emerge un profilo utente con evidenti preferenze consolidate per l'animazione Disney classica. Tuttavia, a causa di alcune raccomandazioni ripetute tra più caroselli (es. *Snow White and the Seven Dwarfs*, *Winnie the Pooh and the Blustery Day*, ...) si è preferito ripetere l'esperimento con un supporto minimo di *min_support* = 100 film, in modo da recuperare feature più importanti.

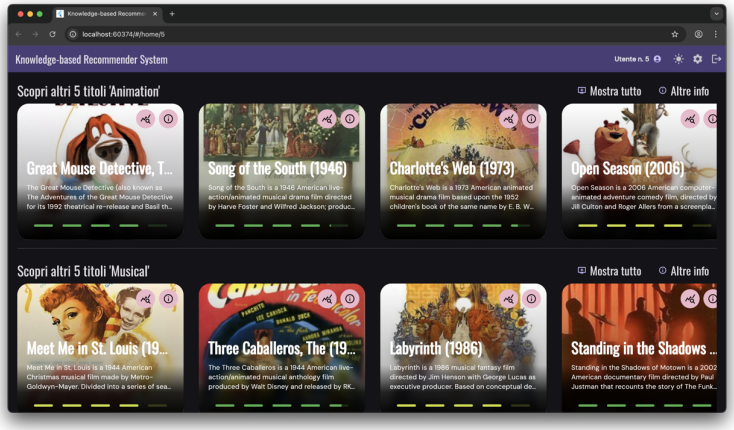


Figura 22: *Caso utente n. 5 - Esempio 2*

Con questa modifica, le nuove top feature raccomandate all'utente 5 sono *Animation* e *Musical*, che comunque si allineano con le precedenti preferenze dell'utente.

L'obiettivo **Novelty** si manifesta in consigli come *Song of the South (ID: 2099)* che è un film sia di genere *Animation* sia di genere *Musical*, adeguato alle preferenze dell'utente.

4.3.3 Utente n. 37

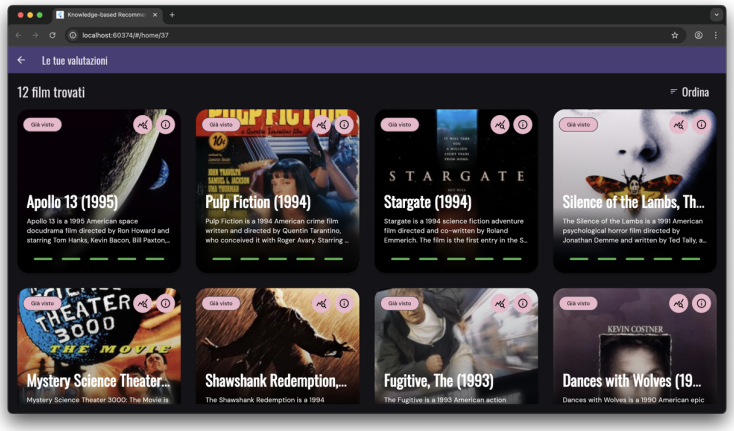


Figura 23: *Caso utente n. 37 - Valutazioni*

Dall'osservazione delle valutazioni di questo utente, si può ipotizzare una marcata preferenza per i film degli anni '90. I titoli con *rating* = 5.0 come *Apollo 13 (ID: 150)*, *Pulp Fiction (ID: 296)* e *Silence of the Lambs (ID: 593)*, mostrano una chiara tendenza verso produzioni di quell'epoca suggerendo che le scelte dell'utente siano orientate verso film usciti in quel decennio.

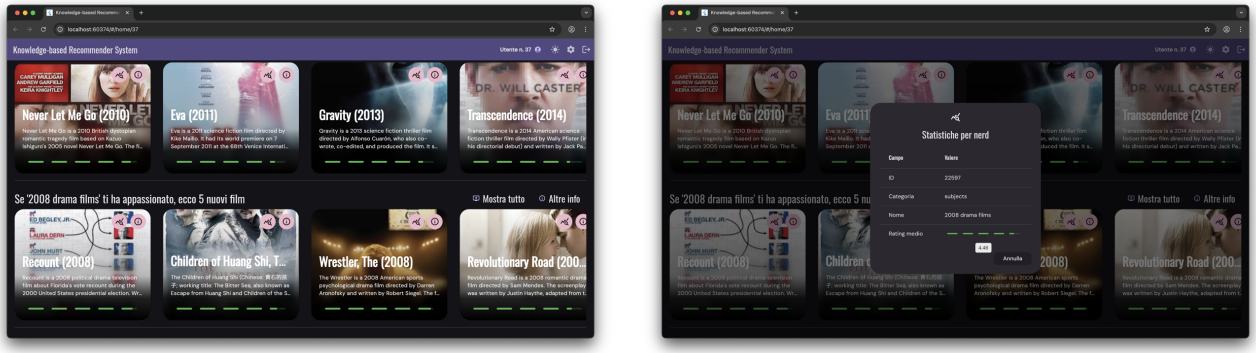


Figura 24: *Caso utente n. 37*

Si potrebbe dire che la **Relevance** è eccellente [5], tuttavia l'analisi delle raccomandazioni per l'utente 37 mette in luce come il sistema, pur offrendo suggerimenti coerenti e con rating elevati, non sempre riesca a fornire motivazioni facilmente interpretabili rispetto alle reali preferenze espresse dall'utente. Mentre dalle valutazioni si può ipotizzare una preferenza per i titoli degli anni '90, le raccomandazioni sono orientate in larga parte verso produzioni più recenti e di generi differenti.

Questa discrepanza può suggerire che il sistema attribuisca elevati rating predetti sulla base di feature meno evidenti, esaltando così caratteristiche che non necessariamente riflettono gusti realmente manifestati. Di conseguenza, il calcolo del rating medio della feature può sovrastimare l'importanza di aspetti subdoli, portando a raccomandazioni apparentemente poco intuitive.

[5] Le feature proposte hanno rating medi molto elevati (tra 4.46 e 4.53) e i film raccomandati presentano rating previsti sempre sopra 4.3, fino a 5.0 (nonostante sia un rating predetto).

5 Valutazione del sistema

La valutazione del sistema di raccomandazione è stata condotta attraverso un esperimento *offline*, il cui scopo primario è di misurare l'efficacia della *predizione dei rating*. L'obiettivo non è di valutare la qualità della lista di raccomandazioni finale (di difficile realizzazione senza il feedback di utenti reali), ma piuttosto di quantificare l'accuratezza del modello nello stimare le abitudini di rating di un utente.

5.1 LensKit

L'intera pipeline sperimentale è stata costruita utilizzando *LensKit* [4], una libreria open-source di Python specificamente progettata per lo sviluppo e la valutazione di sistemi di raccomandazione.

LensKit fornisce strumenti standardizzati per la gestione e la suddivisione dei dati sperimentali. Nel nostro caso, abbiamo utilizzato il modulo `lenskit.crossfold` per implementare una metodologia di validazione hold-out. Attraverso la funzione `partition_users`, ogni utente nel dataset viene processato nel seguente modo: il 20% dei suoi rating viene estratto casualmente per formare il *test set*, mentre il restante 80% costituisce il *training set*.

Inoltre, LensKit include le implementazioni ottimizzate di algoritmi di raccomandazione consolidati, permettendoci di confrontare il nostro modello custom con delle baseline robuste come `ItemItem` (Item-based KNN), `UserUser` (User-based KNN) e `ALS` (Alternating Least Squares con bias).

Infine, LensKit è progettato per essere estendibile con degli algoritmi custom. Sebbene il nostro `CosineSimilarityRecommender` non derivi direttamente dalle classi base di LensKit, l'architettura modulare della libreria ha permesso di integrarlo nella pipeline di valutazione e di confrontarlo con gli altri algoritmi di raccomandazione.

5.2 Pipeline di valutazione

La pipeline implementata nello script `eval.py` è la seguente:

1. **Caricamento e preparazione dei dati**: Lo script inizia caricando:

   - Il dataset dei rating (`EXISTING_RATINGS_PATH`).
   - Il dataset dei film (`EXISTING_MOVIES_PATH`).
   - La matrice sparsa di film e feature (`MOVIE_FEATURE_MATRIX_PATH`).

   Poi i rating vengono normalizzati rinominando le colonne in un formato standard (`user`, `item`, `rating`), compatibile con le convenzioni di LensKit.

2. **Inizializzazione degli algoritmi**: Vengono istanziati cinque algoritmi da confrontare, suddivisi in due categorie: i nostri algoritmi content-based e gli algoritmi collaborative

filtering baseline.

- **I nostri algoritmi content-based**: Il sistema valuta due varianti di `CosineSimilarityRecommender`.
  - **CosineSimilarityRecommender (senza bias)**: replica fedelmente la logica di predizione content-based utilizzata in `build_ratings_complemented.py`. L'algoritmo calcola la matrice di cosine similarity tra tutti i film, basandosi sulle loro feature.
  - **CosineSimilarityRecommender (con bias)** estende la versione base integrando il **bias item** (b_i), cioè la tendenza di un film a ricevere voti sopra o sotto la media globale, calcolato come $b_i = \overline{r}_i - \mu$

    Si calcola quindi una misura di confidenza basata sulla similarità massima tra il film target e i film già valutati dall'utente. Quando le similarità sono elevate, significa che l'utente ha valutato film molto simili a quello target e il sistema si affida maggiormente alla predizione content-based; viceversa, quando le similarità sono basse, aumenta il peso del bias statistico dell'item:

    $$\text{confidence} = \min(1.0, 2 \cdot \max_{j \in RatedItems(u)} \text{sim}(i, j))$$

    $$\hat{r}_{ui} = \hat{r}_{ui}^{\text{base}} + b_i \cdot (1 - \text{confidence})$$

    Questo approccio corregge le predizioni dei rating mancanti adattandosi al contesto: per raccomandazioni "sicure" (film simili a quelli già apprezzati), privilegia l'informazione specifica del contenuto; per raccomandazioni più "esplorative" (film diversi dal profilo abituale), si affida alle tendenze collaborative catturate dai bias.

```python
def predict(self, user, item):
        """
        Predice il rating per una coppia (user, item).
        Usa la logica di build_ratings_complemented.py + bias terms.
        """
        if user not in self.user_profiles:
            return np.nan

        # Converti item a int per sicurezza
        mid = int(item)

        # Verifica che il film esista nella matrice di similarità
        if mid not in self.sim_df.index:
            return np.nan

        # Ottieni il profilo dell'utente (film già valutati)
        user_ratings = self.user_profiles[user]

```

```
        # Se l'utente ha già valutato questo film (non dovrebbe succedere nel test)
        if mid in user_ratings:
            return user_ratings[mid]

        # Film già valutati dall'utente
        rated_movies = list(user_ratings.keys())
        rated_ratings = list(user_ratings.values())

        # Filtra solo i film che esistono nella matrice di similarità
        rated_idx = [m for m in rated_movies if m in self.sim_df.columns]

        if len(rated_idx) == 0:
            # Fallback: media dei voti dell'utente
            return float(np.clip(np.mean(rated_ratings), 1.0, 5.0))

        # Logica di base
        sims = self.sim_df.loc[mid, rated_idx].values
        votes = np.array([user_ratings[m] for m in rated_idx])

        if sims.sum() > 0:
            # Predizione basata su similarità pesata con i voti reali
            pred_base = np.dot(sims, votes) / sims.sum()
        else:
            # Fallback: media dei voti reali
            pred_base = np.mean(rated_ratings) if len(rated_ratings) > 0 else 0.5

        # Aggiunta dei bias terms
        if self.use_bias and self.global_mean is not None:
            # Ottieni i bias
            i_bias = self.item_bias.get(mid, 0.0)

            # Se la similarità è alta, ci fidiamo di più del content-based
            max_sim = sims.max() if len(sims) > 0 else 0.0
            confidence = min(1.0, max_sim * 2.0)  # 0.5 + similarità -> confidenza

            # confidenza alta -> più peso al content-based
            # confidenza bassa -> più peso al bias
            bias_weight = 1.0 - confidence

            pred = pred_base + (i_bias * bias_weight)
        else:
            pred = pred_base
```

```
61
62          # Clipping tra 1 e 5
63          pred = float(np.clip(pred, 1.0, 5.0))
64
65          return pred
```

La scelta di includere una versione con dei bias serve a giustificare le alte prestazioni degli algoritmi di collaborative filtering ottimizzati di LensKit che si vedranno nell'analisi dei risultati.

- **Gli algoritmi collaborative filtering di LensKit**: Per valutare le prestazioni del nostro approccio, utilizziamo tre algoritmi di collaborative filtering implementati nativamente dalla libreria LensKit.

  – **ItemKNN** (*Item-based K-Nearest Neighbors*): è un algoritmo di collaborative filtering item-based che predice i rating basandosi sulla similarità tra item, calcolata sui pattern di valutazione di più utenti. A differenza del nostro approccio content-based, ItemKNN calcola la similarità tra due film osservando quali utenti li hanno valutati e come: due film sono considerati simili se tendono a essere valutati in modo analogo dagli stessi utenti. La predizione avviene identificando i k film più simili (nel nostro caso $k = 50$) al film target tra quelli già valutati dall'utente, e calcolando una media ponderata dei loro rating.

  – **UserKNN** (*User-based K-Nearest Neighbors*): adotta una prospettiva complementare, focalizzandosi sulla similarità tra utenti anziché tra item. L'algoritmo identifica i k utenti più simili ($k = 50$) all'utente target (dove la similarità è calcolata confrontando i pattern di rating sui film in comune) e predice il rating basandosi su come questi utenti simili hanno valutato il film in questione. L'approccio user-based tende a funzionare bene quando gli utenti hanno profili di valutazione densi e coerenti, ma può soffrire di problemi di scalabilità con grandi basi utenti.

  – **ALS** (*Biased Matrix Factorization with Alternating Least Squares*): rappresenta un approccio basato sulla fattorizzazione di matrici. L'algoritmo modella le interazioni utente-item come il prodotto di due matrici di fattori latenti a bassa dimensionalità: $\mathbf{P}$ (fattori utente) e $\mathbf{Q}$ (fattori item), dove ogni utente e ogni film sono rappresentati da un vettore di 50 features latenti. La predizione è data da $\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^T \mathbf{q}_i$, dove i bias terms catturano le tendenze globali e i fattori latenti catturano le interazioni più sottili tra le preferenze dell'utente e le caratteristiche dell'item. L'addestramento avviene minimizzando una variante dell'errore quadratico in 10 iterazioni.

3. **Esecuzione della valutazione**: per ogni algoritmo, la funzione `evaluate_predictions` avvia l'addestramento e la predizione. Gli algoritmi vengono prima addestrati sul training set e poi utilizzati per predire i rating del test set. La funzione gestisce gli algoritmi in due modalità: per il nostro `CosineSimilarityRecommender`, chiama direttamente i metodi

`fit()` e `predict()` da noi definiti; per gli algoritmi di LensKit, utilizza invece il metodo nativo `Recommender.adapt()`.

4. **Calcolo delle metriche**: raccolte tutte le predizioni, lo script calcola due metriche di errore standard per ogni algoritmo:

   - **Mean Absolute Error (MAE)**: misura la media delle differenze assolute tra rating reali e predetti.

   $$\text{MAE} = \frac{1}{|T|} \sum_{(u,i) \in T} |r_{ui} - \hat{r}_{ui}|$$

   - **Root Mean Squared Error (RMSE)**: calcola la radice quadrata della media degli errori al quadrato, penalizzando maggiormente le predizioni che sbagliano di molto.

   $$\text{RMSE} = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (r_{ui} - \hat{r}_{ui})^2}$$

5.3 Risultati Sperimentali

L'esecuzione dello script `eval.py` produce i risultati rappresentati nella Tabella 1.

Algorithm	**MAE**	**RMSE**	**N_predictions**
ALS	0.643	0.836	15277
ItemKNN	0.644	0.840	14705
CosineSimilarityPredictor (with bias)	0.652	0.848	15761
UserKNN	0.662	0.866	15133
CosineSimilarityPredictor (no bias)	0.692	0.890	15761

Tabella 1: *Risultati effettivi della valutazione, ordinati per RMSE crescente.*

Osserviamo che il *CosineSimilarityPredictor (with bias)* mostra un miglioramento significativo rispetto alla versione base senza bias, con una riduzione del MAE del *5.8%* (da 0.692 a 0.652) e una riduzione dell'RMSE del *4.7%* (da 0.890 a 0.848). Nonostante il modello base si dimostri il meno performante di tutti, si nota quanto le prestazioni degli algoritmi di collaborative filtering dipendano da correzioni statistiche, assenti nei modelli puramente content-based.

Per quanto riguarda i modelli di collaborative filtering baseline, *ALS* e *ItemKNN* mostrano le prestazioni migliori in assoluto. Questo risultato è atteso: gli algoritmi di collaborative filtering riescono a catturare informazioni sugli utenti e sui film che non sono rilevabili dal solo contenuto dei film stessi. Inoltre, bisogna tenere conto del fatto che questi algoritmi presentano anche ottimizzazioni statistiche, simili a quella del bias da noi proposta.

6 Conclusioni

Oltre alle limitazioni esplorate nella sezione 5, è fondamentale riflettere sulle potenzialità del sistema e sulle prospettive di ricerca correlate.

Questa sezione propone una visione d'insieme delle conclusioni raggiunte durante il progetto. Verranno discusse sia soluzioni tecnologiche che aspetti metodologici utili per superare le criticità emerse, con particolare attenzione all'integrazione di nuove fonti dati, al potenziamento dei meccanismi di personalizzazione e all'adozione di modelli di raccomandazione più avanzati.

6.1 Sintesi dei Risultati

Il progetto ha portato alla realizzazione di un'architettura *client-server* completa, composta da un *backend* Python responsabile della pipeline di elaborazione dati e da un *client* Flutter per l'interazione utente. La pipeline di acquisizione dati arricchisce il dataset di base, estrae e struttura i metadati, quali attori, registi e generi, successivamente trasformati in una matrice *film-feature* per il calcolo della similarità tramite *cosine similarity*. È stato inoltre implementato un sistema di profilazione utente *content-based* e un meccanismo di raccomandazione basato sulla strategia MAB *Softmax exploration*, utile per generare suggerimenti pertinenti ma non monotoni.

La valutazione sperimentale *offline*, condotta tramite il framework LensKit, confronta il modello *content-based* (con e senza bias) con algoritmi di *collaborative filtering*. I risultati hanno mostrato che l'introduzione di un bias statistico migliora significativamente le prestazioni del modello *content-based*. Come previsto, gli algoritmi di *collaborative filtering* hanno evidenziato un'accuratezza predittiva superiore, confermando la loro capacità di catturare informazioni latenti nelle interazioni utente-item non rilevabili dal solo contenuto.

6.2 Limitazioni e sviluppi futuri

Nonostante il raggiungimento degli obiettivi progettuali, si elencano i limiti e le possibili direzioni di sviluppo futuro:

1. **Valutazione e metriche di performance**
   L'analisi si è focalizzata sull'accuratezza della *predizione dei rating* (misurata tramite MAE e $RMSE$), senza considerare metriche di *ranking* (es. $Precision@K$, $nDCG$) utili a valutare la qualità delle liste raccomandate. Inoltre, non sono state impiegate metriche *beyond-accuracy* (quali diversità, novità o serendipity), che consentirebbero di misurare la capacità del sistema di proporre contenuti nuovi e inaspettati. Gli sviluppi futuri dovranno quindi estendere la pipeline di valutazione *offline* per includere tali metriche, e successivamente validare le prestazioni attraverso sperimentazioni *online* (es. A/B testing), in modo da analizzare anche aspetti soggettivi come soddisfazione, fiducia ed *engagement*.

2. **Estensione tramite tecniche di Machine Learning**
   Sebbene siano stati raccolti dati testuali come gli *abstract* dei film e il file `tags.csv` del dataset MovieLens (che contiene descrizioni soggettive ad alto livello semantico), tali

informazioni non sono state utilizzate nella costruzione della matrice *film-feature*. L'integrazione di questi dati richiederebbe l'uso di tecniche avanzate di *Natural Language Processing* e *Machine Learning* (ad esempio, con il *sentiment analysis*) che permetterebbero di catturare relazioni più profonde tra i contenuti dei film e le valutazioni degli utenti. Alla luce dei miglioramenti ottenuti con i bias statistici, prevediamo che un approccio ibrido che combini metadati strutturati, analisi statistiche e rappresentazioni apprese tramite modelli linguistici potrebbe migliorare la qualità e la personalizzazione delle raccomandazioni.

Riferimenti bibliografici

[1] GroupLens Research, “Movielens datasets,” 2025. Accessed: 20 October 2025.

[2] CaseRec, “Datasets-for-recommender-systems: A repository of public data sources for recommender systems,” 2025. Accessed: 21 October 2025.

[3] DBpedia Association, “Dbpedia,” 2025. Accesso: 21 ottobre 2025.

[4] M. D. Ekstrand, J. A. Konstan, and J. Riedl, “Lenskit: Open-source recommender systems framework,” 2025. Accessed: 21 October 2025.